TaskFlow – Analyse fonctionnelle et conception d'une application de gestion de projets

1 - Contexte et objectifs du projet 4
1.1 - Livrables API 5
1.2 - Livrables Swift 5
1.3 Code source 5
2 - Environnement technique et outils utilisés 6
3 - Stratégie de conduite du projet 8
3.1 Approche fonctionnelle et stratégique 8
3.2 Découpage du planning par semaine 9
3.4 Utilisation de Trello pour le suivi du projet 11
3.5 Détail des user stories et checklists d'actions 13
3.6 - Note sur l'organisation du projet 14
4 - Analyse fonctionnelle 15
4.1 Glossaire et conventions utilisées 15
4.2 - Cas d'utilisation 16
4.3 - Spécification fonctionnelle : user stories 17
4.3.1. Authentification et gestion des profils 17
4.3.2. Gestion des projets 20
4.3.3. Gestion des user stories 22
4.3.4. Gestion des tâches 24
4.3.5. Gestion des tags 26
4.4 - Priorisation des fonctionnalités et estimation de la complexité 27
4.4.1 - Priorités 27
4.4.2 - Complexité 29
5 - Conception du système 30
5.1 - Modèle de données 30
5.2 - Diagramme de classes 32
5.3 - Diagramme des contrôleurs 34
6 - Architecture logicielle et choix techniques 36
6.1 Architecture générale de l'application 36
6.2 Organisation des contrôleurs et des routes API 37

6.3 Gestion des formulaires et validation des données....38
6.4 - Sécurité et gestion des accès....39
6.4.1 - Authentification (JWT, refresh Token)....39
6.4.2 - Gestion des rôles et profils utilisateurs....40
6.4.3 - Matrice des autorisations et règles d'accès....41
6.4.4 Implémentation des contrôles d'accès....44
6.4.5 Traçabilité et cohérence avec les tests....45
6.5 Règles métier et gestion des statuts....46
6.6 - Intégration des méthodes agiles : notion de User Story....47
7 - Tests et validation....48
7.1 Stratégie de tests....48
7.2 Organisation des tests....49
7.3 Couverture fonctionnelle....50
7.4 - Detail des tests....51
7.4.1 - Utilisateurs....51
7.4.2 Administrateurs / Managers....53
7.4.4 - User stories....57
7.4.5 - Tâches: TaskTest.php....59
7.4.6 - Tags: TagTest.php....61
7.5 - Résultat des tests....64
8 – Application iOS (Swift)....65
8.1 Objectif de l'application mobile....65
8.2 - Architecture de l'application....66
8.2.1 - Model....66
8.2.2 - Cinématique....67
8.2.3 - Views....71
8.2.4 - ViewModel....72
8.3 - Gestion de la navigation....73
8.4 - Communication avec le Backend....74
8.4.1 - APIClient....74
8.4.2 - Services réseau....75
8.4.3 - Exemple d'appel API....76
8.5 - Sécurité et authentification....77
8.5.1 - JWT....77

8.5.2 - SessionManager 78
8.5.3 - Biométrie (Face ID / Touch ID) 79
8.6 – Choix techniques 80
8.6.1 – Architecture MVVM 80
8.6.2 – Gestion de la navigation 81
8.6.3 – Programmation asynchrone 82
8.6.4 – Adaptation multi-appareils 83
8.6.5 – Centralisation des constantes UI 84
8.6.6 – Gestion des thèmes clair / sombre 85
8.6.7 – Expérience utilisateur et interactions 86
8.7 - Liens Backend - iOS 87
8.8 – Description des écrans 88
8.8.1 – Authentification 88
8.8.2 – Backlog 96
8.8.3 – Utilisateurs 112
8.8.4 – Projets 120
8.8.5 - Gestion des mots de passe 132
9 – Tests XCTest 138
9.1 – Tests de la connexion 139
9.2 - Tests des mots de passe 140
9.3 - Tests Projets 141
9.4 - Tests User Stories 142
9.5 - Tests Session 143
9.6 - Résultats des tests 144

1 - Contexte et objectifs du projet

La startup TaskFlow souhaite lancer une application mobile pour organiser ses projets en mode agile.

Elle vous confie le développement d'une API avec Symfony, l'authentification JWT, la gestion des rôles et autorisations, la documentation, les tests ainsi que la préparation au déploiement.

Vous intégrerez également la gestion de projet agile (Scrum-ban), l'utilisation de Git/GitHub et de GitHub Projects/Trello pour planifier et suivre le travail.

1.1 - Livrables API

Livrables

- Code source sur GitHub (README : prérequis, installation, scripts, variables d'environnement, comptes de démo).
- Schémas : cas d'utilisation et modèle de données (UML/mermaid), matrice d'autorisations (rôles x actions).
- Documentation API : Swagger.
- Plan de tests : liste, jeux d'essai, rapport d'exécution (unitaires/fonctionnels).
- Gestion de projet : export/captures du board, CR de rituels, burndown/velocity.

1.2 - Livrables Swift

Livrables

- Code source sur GitHub avec un README (installation, lancement, variables d'environnement API).
- Maquettes d'écrans (Figma ou Xcode/SwiftUI Previews) avec navigation entre pages.
- Documentation technique (architecture, state management, sécurité des données).
- Tests unitaires et rapport de couverture.
- Build iOS exécutable pour démonstration.

1.3 Code source

Le code source du projet est disponible sur GitHub :
https://github.com/dgztsdmnsd-blip/taskflow

L'API backend est déployée en production à l'adresse suivante :
https://lucban.alwaysdata.net

La documentation de l'API (Swagger / OpenAPI) est accessible à :
https://lucban.alwaysdata.net/api/doc

Fichier ReadMe sur GitHub :
https://github.com/dgztsdmnsd-blip/taskflow

2 - Environnement technique et outils utilisés

Spécifications techniques

- Stack : PHP 8.2+, Symfony, Doctrine ORM, PostgreSQL ou MySQL.
- API : REST/JSON ; pagination, tri, filtres (par état, priorité, échéance, tag, membre).
- Sécurité : JWT (login, refresh), mots de passe hachés, validation systématique des données, rôles et voters pour le contrôle d'accès, CORS restrictif (origines mobiles autorisées), rate limiting sur /auth, messages d'erreur sobres.
- Qualité & tests : PHPUnit, tests unitaires (services, voters), fonctionnels (contrôleurs), jeux d'essai (fixtures), linting (PHP-CS-Fixer), analyse statique (PHPStan/Psalm).
- Documentation : Swagger.
- Erreurs & logs : format JSON unifié (code, message, details), statuts HTTP cohérents, journalisation (Monolog).
- RGPD : politique de mots de passe.

Outils:

Outil	Usage	Instllation / Accès
Visual Studio Code	Éditeur de code principal utilisé pour le développement de l'application (PHP, Symfony, tests)	Téléchargement depuis le site officiel
Postman	Tests manuels des routes API REST (requêtes HTTP, authentification JWT, validation des réponses)	Téléchargement depuis le site officiel
Trello	Organisation et suivi du projet (user stories, avancement, priorités)	Application web
xDiagram	Création de diagrammes UML (diagrammes de classes, de séquence, de contrôleurs)	Application web
DBeaver	Gestion et visualisation de la base de données (requêtes SQL, inspection des tables)	Téléchargement depuis le site officiel
Pages	Rédaction de la documentation technique et fonctionnelle du projet	Application macOS
Symfony	Framework PHP utilisé pour le développement de l'API REST	
Symfony MakerBundle	Génération automatisée de code (entités, contrôleurs, formulaires, tests)	composer require --dev symfony/maker-bundle
Doctrine ORM	Gestion de la persistance des données et mapping objet-relationnel	composer require orm
NelmioApiDocBundle	Gestion de la documentation des routes de l'API	composer require nelmio/cors-bundle
Symfony Serializer	Sérialisation et désérialisation des données JSON	composer require symfony/serializer
LexikJWTAuthenticationBundle	Mise en place de l'authentification par JWT	composer require lexik/jwt-authentication-bundle

Outil	Usage	Instllation / Accès
GitHub	Versionnement du code source et gestion du dépôt distant	Plateforme web
PHPUnit	Tests unitaires et fonctionnels automatisés	Inclus avec Symfony
Symfony Fixtures	Création de l'utilisateur par défaut	*composer require orm-fixtures*
Xcode	Developpement iOS / SwitUI	Version 26.2
Zenstruck Foundry	Génération de jeux de données de test et facilitation des tests	composer require --dev zenstruck/foundry

3 - Stratégie de conduite du projet

3.1 Approche fonctionnelle et stratégique

La stratégie de ce projet a été définie à partir d'une analyse fonctionnelle des besoins utilisateurs, formalisée sous forme de **user stories**. Cette approche permet de garantir une adéquation entre les fonctionnalités développées et les usages attendus, tout en structurant le projet de manière progressive et cohérente.

Dans un premier axe, le projet se concentre sur la **gestion de l'authentification et des profils utilisateurs**. Cette étape constitue un socle fondamental, visant à assurer la sécurité des accès, la protection des données personnelles et la gestion des rôles (utilisateur, manager / administrateur). La mise en place de règles strictes de validation, de chiffrement des mots de passe et de gestion des sessions permet de répondre aux exigences de fiabilité et de sécurité de l'application.

Le deuxième axe stratégique porte sur la **gestion des projets**, en offrant aux managers des fonctionnalités leur permettant de créer, administrer et organiser des projets collaboratifs. Chaque projet est associé à des règles d'accès précises, garantissant que seules les personnes autorisées puissent consulter ou modifier les informations liées au projet.

Le cœur fonctionnel de l'application repose sur la **gestion des user stories et des tâches**, qui permet de décomposer les besoins métiers en éléments opérationnels. L'utilisation d'un workflow structuré (non démarrée, en cours, terminée) facilite le suivi de l'avancement, la coordination entre les membres du projet et la traçabilité des actions effectuées.

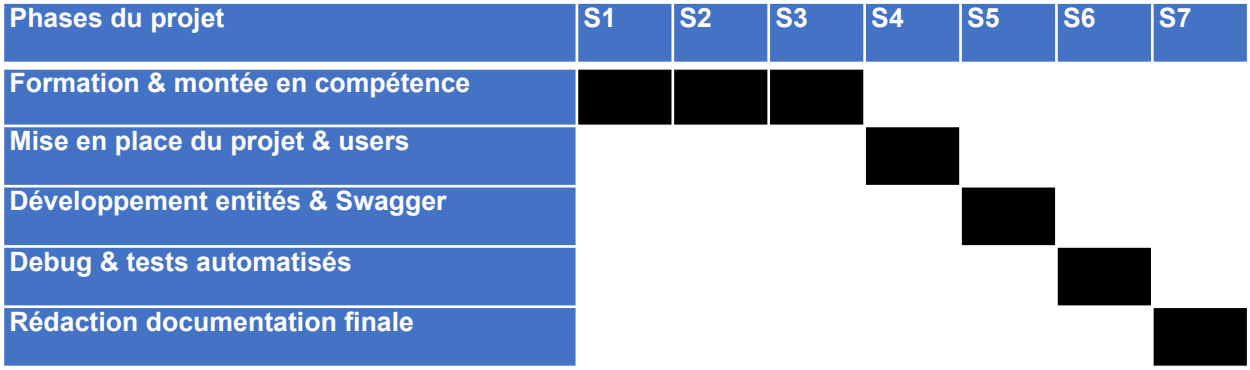
Enfin, la **gestion des tags** vient compléter cette stratégie en proposant un mécanisme de classification et de filtrage du contenu. Cette fonctionnalité améliore l'organisation des informations et contribue à une meilleure lisibilité des projets et de leurs éléments constitutifs.

Ainsi, la stratégie globale du projet vise à concevoir une application sécurisée, structurée et évolutive, répondant à des objectifs pédagogiques et techniques, tout en respectant les bonnes pratiques de conception logicielle et de gestion de projet.

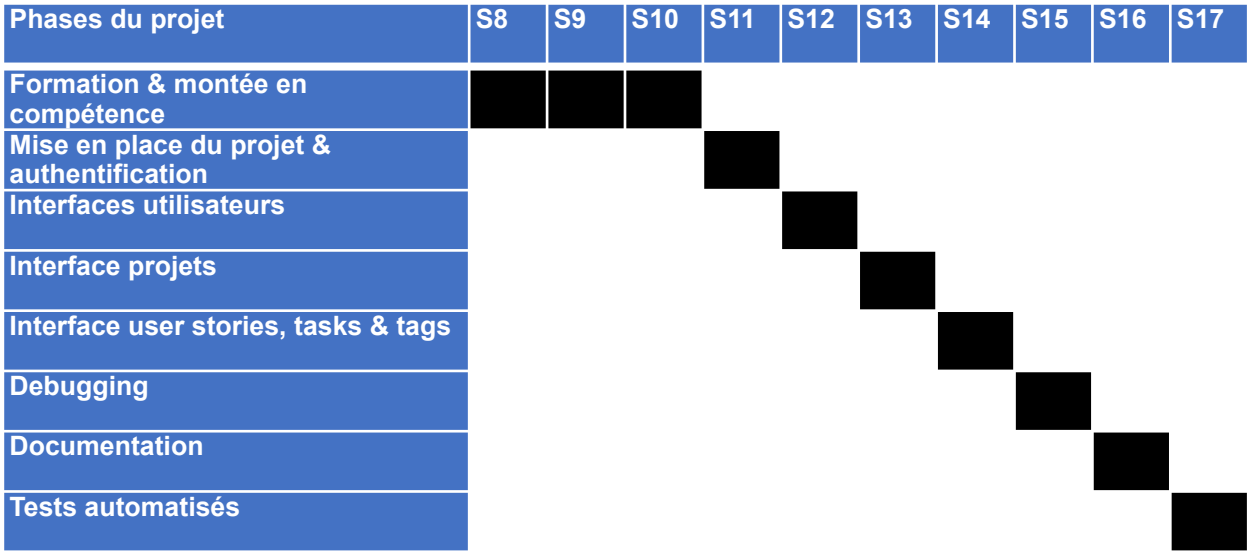
3.2 Découpage du planning par semaine

Le diagramme de Gantt ci-dessous présente la planification temporelle du projet sur une durée de **sept semaines**, en mettant en évidence les différentes phases de travail et leur enchaînement.

Partie API:



Partie iOS



Lecture du diagramme

- Les semaines 1 à 3 sont dédiées à la formation et à l'appropriation des technologies.
- La semaine 4 marque la mise en place du socle technique (architecture, utilisateurs, authentification).
- La semaine 5 est consacrée au développement des entités métier et à la documentation de l'API.
- La semaine 6 est dédiée au déboggage et aux tests automatisés.
- La semaine 7 est réservée à la rédaction de la documentation finale et à la préparation des livrables.
- Les semaines 8 à 10 sont dédiées à la formation Swift.
- La semaine 11 marque le début du projet iOS avec la mise en place de l'application et de la mise en place de la connexion utilisateur.
- La semaine 12 est utilisée pour la mise en place des interfaces utilisateurs.
- La semaine 13 est consacrée à la mise en place des projets dans l'application.
- La semaine 14 est dédié à la mise en place des entités user stories, tasks et tags.
- La semaine 15 est utilisée pour le debugging et finalisation de l'application.
- La semaine 16 est consacrée à la documentation.
- La semaine 17 sera liée à la mise en place des tests automatiques sur xCode.

3.4 Utilisation de Trello pour le suivi du projet

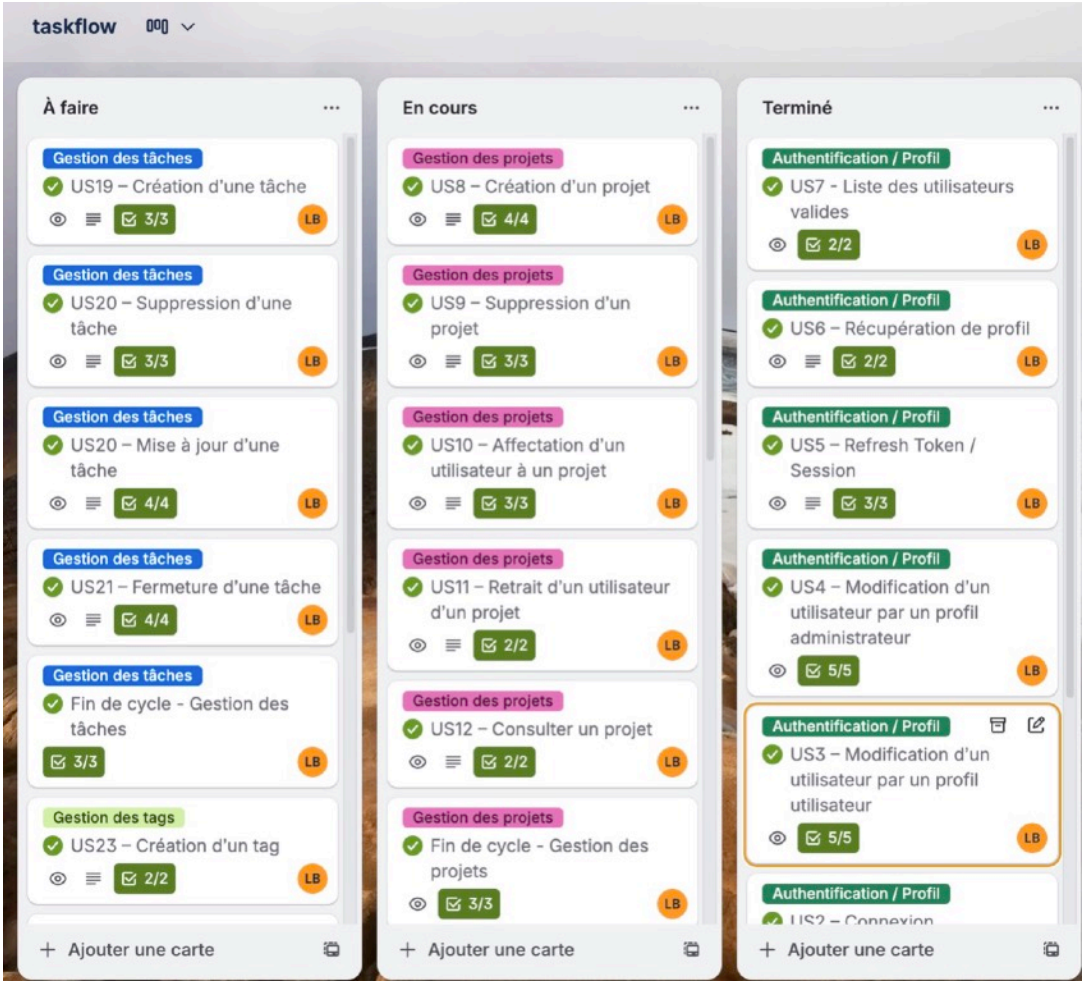
Afin d'assurer un suivi structuré et visuel de l'avancement du projet, l'outil **Trello** a été utilisé comme support de gestion de projet.

Le projet a été découpé en **user stories** et **fonctionnalités**, organisées sous forme de cartes réparties dans des colonnes représentant l'état d'avancement des tâches : *À faire*, *En cours* et *Terminé*.

Chaque carte correspond à une fonctionnalité ou une User Story identifiée, permettant :

- de suivre l'avancement du développement,
- de visualiser les tâches terminées,
- de prioriser les actions à réaliser,
- d'assurer une meilleure organisation du travail.

La figure ci-dessous illustre le tableau Trello utilisé durant le projet, mettant en évidence le découpage fonctionnel et le suivi des différentes étapes de développement.



3.5 Détail des user stories et checklists d'actions

Afin de garantir une mise en œuvre structurée et progressive des fonctionnalités, chaque **User Story** du projet a été accompagnée d'une **checklist détaillant les actions à mener**.

Ces checklists permettent de découper chaque User Story en tâches techniques précises, facilitant ainsi :

- l'estimation de la charge de travail,
- le suivi de l'avancement,
- la validation complète de la fonctionnalité,
- la réduction des oublis lors du développement.

Les actions listées correspondent notamment à :

- la création ou la mise à jour des entités,
- le développement des contrôleurs et des routes API,
- la gestion des règles de sécurité et des droits d'accès,
- l'écriture des tests associés,
- la vérification via des tests manuels (Postman).

Chaque action est validée individuellement dans Trello au fur et à mesure de son implémentation, permettant d'assurer qu'une User Story n'est considérée comme terminée que lorsque l'ensemble de ses critères techniques est respecté.

Cette approche contribue à une meilleure maîtrise du projet et s'inscrit pleinement dans une démarche inspirée des méthodes agiles.

3.6 - Note sur l’organisation du projet

Ce projet ayant été réalisé individuellement, certains livrables classiquement associés à un travail d’équipe (comptes rendus de rituels Scrum, velocity, burndown chart, etc.) ne sont pas applicables.

L’organisation du travail a néanmoins été structurée à l’aide de Trello, avec une planification hebdomadaire et un suivi des tâches.

4 - Analyse fonctionnelle

4.1 Glossaire et conventions utilisées

Afin d'assurer une compréhension homogène des termes employés tout au long de ce document, les conventions suivantes sont appliquées.

Statut des utilisateurs

- **ACTIVE** : utilisateur autorisé à se connecter à l'application et à accéder aux fonctionnalités correspondant à son profil.
- **INACTIVE** : utilisateur désactivé, ne pouvant plus se connecter ni interagir avec l'application.

Profils utilisateurs

- **Utilisateur** : membre d'un ou plusieurs projets, disposant d'un accès aux fonctionnalités opérationnelles (consultation et mise à jour des éléments auxquels il est autorisé).
- **Manager / Administrateur** : utilisateur disposant de droits étendus, notamment la création et la gestion des projets, la gestion des membres et la modification des statuts.

Statuts des projets, des user stories et des tâches

- **NS (Non démarré)** : élément créé mais dont le traitement n'a pas encore commencé.
- **ENC (En cours)** : élément actuellement en cours de réalisation.
- **TER (Terminé)** : élément finalisé, validé et ne nécessitant plus d'action.

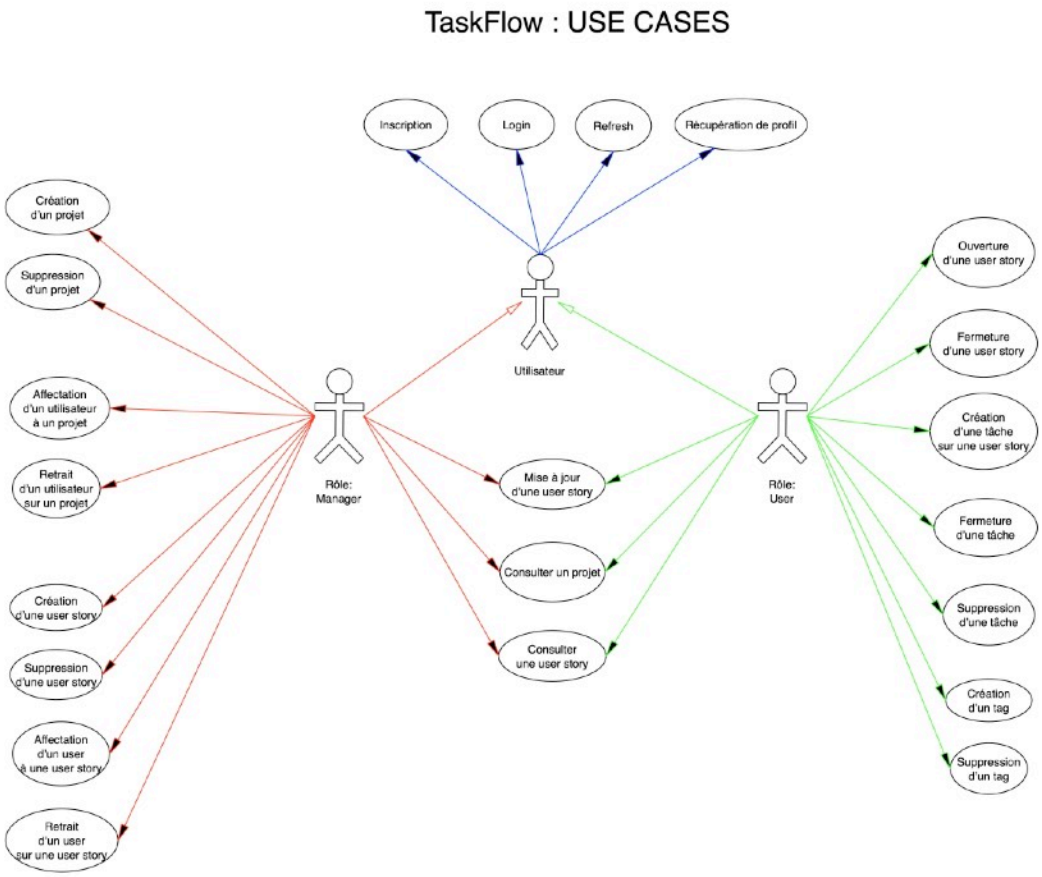
Ces conventions sont utilisées de manière uniforme dans l'analyse fonctionnelle, la conception du système, les règles métier et les scénarios de tests.

4.2 - Cas d'utilisation

Le diagramme de cas d'utilisation présenté ci-dessous illustre les principales interactions entre les acteurs du système et les fonctionnalités de l'application.

Il met en évidence les différents rôles (utilisateur, manager / administrateur) ainsi que les actions auxquelles chacun peut accéder. Chaque groupe de cas d'utilisation correspond à un domaine fonctionnel précis, notamment l'authentification, la gestion des projets, des user stories, des tâches et des tags.

Ce diagramme permet d'avoir une vision globale du périmètre fonctionnel du projet et de clarifier les responsabilités de chaque acteur au sein du système.



4.3 - Spécification fonctionnelle : user stories

4.3.1. Authentification et gestion des profils

US1 – Inscription

En tant qu'utilisateur, je veux créer un compte afin d'accéder à l'application.

Critères d'acceptation :

- L'utilisateur doit fournir son adresse e-mail et son mot de passe.
- L'adresse e-mail doit être unique.
- Le format de l'email doit être valide.
- Le mot de passe doit être crypté.
- Le compte est créé si toutes les conditions sont remplies.
- Le mot de passe doit avoir 8 char minimum, au moins 1 chiffre, au moins 1 majuscule et 1 caractère spécial.
- Pas besoin d'être identifié pour créer un compte

US2 – Connexion

En tant qu'utilisateur, je veux me connecter afin d'utiliser mes fonctionnalités personnelles.

Critères d'acceptation :

- L'utilisateur fournit email + mot de passe.
- Si les identifiants sont valides, la connexion réussit.
- Si incorrect, un message d'erreur s'affiche.
- Un utilisateur ne peut pas se connecter si son compte est inactif

US3 – Modification d'un utilisateur par un utilisateur

En tant qu'utilisateur, je veux modifier mes informations personnelles ou mon mot de passe.

Critères d'acceptation :

- L'utilisateur doit être authentifié.
- L'utilisateur modifié doit être celui connecté.
- L'utilisateur peut modifier prénom, nom, email, mot de passe.

- Les Champs doivent être valide selon US1.
- Les données sont enregistrés en base.

US4 – Modification d'un utilisateur par un Manager

En tant que manager, je veux modifier le statut ou le rôle d'un utilisateur.

Critères d'acceptation :

- L'utilisateur doit être authentifié.
- Le manager peut modifier le statut et le role de tout utilisateur
- Le role peut être uniquement MGR ou UTIL
- Le profil peut être uniquement ACTIVE ou INACTIVE
- Les données sont enregistrés en base.

US5 – Refresh Token / Session

En tant qu'utilisateur, je veux que ma session puisse être rafraîchie afin d'éviter de me reconnecter toutes les dix minutes.

Critères d'acceptation :

- Le système renouvelle automatiquement la session avant expiration.
- Le renouvellement est transparent pour l'utilisateur.
- L'utilisateur n'est pas déconnecté avant un délai raisonnable.

US6 – Récupération de profil

En tant qu'utilisateur, je veux récupérer les informations de mon profil afin de les consulter ou les utiliser dans l'application.

Critères d'acceptation :

- L'utilisateur connecté peut récupérer ses informations.
- Les données renvoyées sont complètes et exactes.

US7 – Liste des utilisateurs connectés

En tant qu'utilisateur, je veux lister les utilisateurs de l'application afin consulter les membres de l'équipe ou d'intervenir sur les droits si je suis un manager.

Critères d’acceptation :

- L'utilisateur doit être connecté
- La liste renvoyée contient uniquement les utilisateurs avec un statut ACTIVE.

4.3.2. Gestion des projets

US8 – Création d'un projet

En tant que manager, je veux créer un projet afin d'y organiser mon travail et celui de mon équipe.

Critères d'acceptation :

- Le manager peut créer un projet.
- Un nom de projet est obligatoire.
- Le projet apparaît dans la liste des projets.
- Le créateur est automatiquement l'owner.

US9 – Suppression d'un projet

En tant que manager, je veux supprimer un projet afin de retirer de l'application ceux qui ne sont plus pertinents.

Critères d'acceptation :

- Seul le owner du projet peut le supprimer.
- Le projet est retiré de la liste.
- Les données associées au projet sont soit supprimées, soit archivées en fonction des règles métier.

US10 – Affectation d'un utilisateur à un projet

En tant que manager, je veux affecter un utilisateur à un projet afin qu'il puisse y travailler.

Critères d'acceptation :

- Le manager peut sélectionner un utilisateur existant.
- L'utilisateur apparaît dans la liste des membres du projet.
- L'utilisateur voit le projet dans son espace.

US11 – Retrait d'un utilisateur d'un projet

En tant que manager, je veux retirer un utilisateur d'un projet afin qu'il n'y ait plus accès.

Critères d'acceptation :

- Le manager peut retirer un utilisateur du projet.
- L'utilisateur ne voit plus le projet.

US12 – Consulter un projet

En tant qu'utilisateur, je veux consulter un projet afin de voir ses informations et son contenu.

Critères d'acceptation :

- L'utilisateur voit : nom, description, date, membres, user stories.
- L'accès doit être refusé si vous n'êtes pas membre du projet.

4.3.3. Gestion des user stories

US13 – Création d'une User Story

En tant que manager, je veux créer une User Story afin de décrire un besoin fonctionnel lié au projet.

Critères d'acceptation :

- Le manager peut créer une User Story avec titre obligatoire.
- L'US est visible dans la liste du projet.
- L'état de la User Story est « Non démarrée ».

US14 – Suppression d'une User Story

En tant que manager, je veux supprimer une User Story afin de nettoyer celles qui ne sont plus nécessaires.

Critères d'acceptation :

- Seul le owner de la User Story peut la supprimer.
- Confirmation nécessaire.
- Les tâches associées sont gérées selon la règle métier (supprimer / orphelines).

US15 – Mise à jour d'une User Story

En tant que manager, je veux modifier le contenu d'une User Story afin de mettre à jour le besoin.

Critères d'acceptation :

- Le manager peut modifier le titre, description, statut, priorité.
- Les valeurs modifiées sont enregistrées et visibles.

US16 – Consulter une User Story

En tant qu'utilisateur, je veux consulter une User Story afin de comprendre le besoin et son état.

Critères d'acceptation :

- L'utilisateur voit les informations complètes.
- Accès interdit si non membre du projet.

US17 – Démarrer une User Story

En tant qu'utilisateur, je veux changer statut d'une User Story à « En cours » afin d'indiquer le début des travaux.

Critères d'acceptation :

- Seul un membre du projet peut changer le statut selon workflow valide.
- Seule une User Story « Non démarrée » peut être ouverte.
- L'historique / activité est mis à jour.

US18 – Terminer une User Story

En tant qu'utilisateur, je veux changer statut d'une User Story à « Terminé » afin d'indiquer la fin des travaux.

Critères d'acceptation :

- Seul un membre du projet peut changer le statut selon workflow valide.
- Seule une User Story « En cours » peut être terminée.
- L'historique / activité est mis à jour.

4.3.4. Gestion des tâches

US19 – Création d'une tâche

En tant qu'utilisateur, je veux créer une tâche liée à une User Story afin de découper le travail.

Critères d'acceptation :

- L'utilisateur peut créer une tâche avec un titre obligatoire.
- Seul un membre du projet peut créer une tâche.
- La tâche est rattachée à une User Story.

US20 – Mise à jour d'une tâche

En tant que owner de la User Story, je veux modifier une tâche liée afin de corriger le contenu.

Critères d'acceptation :

- Champs modifiés
- Enregistrer les champs
- Changements visibles
- Changements instantanés

US21 – Fermeture d'une tâche

En tant qu'utilisateur, je veux fermer une tâche afin de signaler qu'elle est terminée.

Critères d'acceptation :

- L'utilisateur peut passer la tâche en « Terminé ».
- Seul le owner peut terminer une tâche.
- Seul une tâche ouverte peut être terminée.
- La date de clôture est enregistrée.

US22 – Suppression d'une tâche

En tant qu'utilisateur, je veux supprimer une tâche afin de retirer celles qui n'ont plus lieu d'être.

Critères d'acceptation :

- Seul le owner peut supprimer une tâche.

- Une confirmation est demandée.
- La tâche disparaît définitivement.

4.3.5. Gestion des tags

US23 – Création d'un tag

En tant qu'utilisateur, je veux créer un tag afin de catégoriser ou filtrer le contenu.

Critères d'acceptation :

- Seul un membre du projet peut créer un tag avec un nom unique.
- Le tag apparaît dans la liste des tags disponibles.

US24 – Affectation d'un tag

En tant qu'utilisateur, je veux ajouter un tag existant afin de catégoriser ou filtrer le contenu.

Critères d'acceptation :

- Seul un membre du projet peut ajouter un tag existant.
- Le tag n'est pas créé en double dans la liste des tags disponibles.

US25 – Suppression d'un tag

En tant qu'utilisateur, je veux supprimer un tag afin qu'il n'apparaisse plus dans le système.

Critères d'acceptation :

- Seul un membre du projet peut supprimer un tag existant.
- Les objets associés perdent ce tag.

US26 – Rechercher user stories portant un tag

En tant qu'utilisateur, je veux filtrer la lister des user stories portant un tag précis afin d'organiser mon travail.

Critères d'acceptation :

- Retourne uniquement les objets associés à ce tag.
- L'utilisateur doit être authentifié.
- Seuls les projets où l'utilisateur connecté est membre sont retournés.

4.4 - Priorisation des fonctionnalités et estimation de la complexité

4.4.1 - Priorités

Fonctionnalités	Priorité	Complexité
Authentification / Profil	Must Have	Elevée
Gestion des projets	Must Have	Moyenne
Gestion des user stories	Must Have	Elévée
Gestion des tâches	Should Have	Moyenne
Gestion des tags	Must Have	Faible
Gestion des mots de passe oubliés	Nice to Have	Moyenne

Dans le cadre de ce projet, les fonctionnalités ont été priorisées en fonction des exigences du cahier des charges initial, qui imposait la gestion des utilisateurs, l'authentification par token ainsi que la gestion des projets, des tâches et des tags. Afin d'organiser le développement de manière progressive et maîtrisée, une approche de priorisation de type **Must Have / Should Have / Nice to Have** a été adoptée.

Les fonctionnalités liées à **l'authentification et à la gestion des profils utilisateurs** ont été classées en **Must Have**, car elles constituent le socle sécuritaire de l'application. Elles assurent la gestion des rôles, la validation des statuts (ACTIVE / INACTIVE), la protection des accès via JWT et le rafraîchissement des sessions. Sans ce module, aucune autre fonctionnalité ne pourrait être exploitée de manière fiable.

La **gestion des projets**, des **user stories** et des **tags** a également été positionnée en **Must Have**. Ces éléments structurent l'organisation fonctionnelle du système :

- les projets constituent les espaces de travail collaboratifs,
- les user stories permettent de formaliser les besoins métiers et d'organiser le backlog,
- les tags facilitent la classification et le filtrage des contenus.

Ces fonctionnalités forment l'ossature métier principale de l'application.

La **gestion des tâches** a été classée en **Should Have**. Bien qu'essentielle au suivi opérationnel, elle repose sur des mécanismes déjà établis (projets, user stories, règles d'accès). Elle enrichit le système en apportant un niveau de granularité supplémentaire dans l'organisation du travail, sans constituer le fondement structurel de l'application.

Enfin, la **gestion des mots de passe oubliés** a été positionnée en **Nice to Have**. Cette fonctionnalité améliore l'expérience utilisateur et la robustesse globale du système, mais n'est pas indispensable au fonctionnement principal de l'application.

Cette priorisation a permis d’assurer une progression logique du développement : consolider d’abord les fondations sécuritaires et structurelles, puis enrichir progressivement l’application avec des fonctionnalités complémentaires. Elle garantit ainsi la conformité aux exigences initiales tout en assurant cohérence technique et valeur pédagogique.

4.4.2 - Complexité

La **gestion de l'authentification et des profils utilisateurs** présente une **complexité élevée**, car elle implique plusieurs aspects sensibles tels que la sécurité des données, le chiffrement des mots de passe, la gestion des tokens, le rafraîchissement de session ainsi que le contrôle des rôles et des statuts des utilisateurs.

La **gestion des projets** est évaluée comme ayant une **complexité moyenne**, dans la mesure où elle nécessite la mise en place de relations entre utilisateurs et projets, des règles d'accès spécifiques (owner, membres), ainsi que des opérations de création, consultation et suppression, sans toutefois impliquer de mécanismes techniques particulièrement complexes.

La **gestion des user stories**, ajoutée dans une logique d'inspiration agile, présente une **complexité moyenne**. Elle repose principalement sur des opérations CRUD et sur un workflow simple de statuts, tout en servant de lien fonctionnel entre les projets et les tâches.

La **gestion des tâches** est considérée comme **complexe**, car elle constitue un élément central du suivi opérationnel. Elle implique des contraintes métier fortes (droits selon le owner de la User Story, états autorisés, dépendance aux user stories), une gestion fine des statuts et un impact direct sur l'avancement du projet.

Enfin, la **gestion des tags** a été classée comme ayant une **complexité faible**, car elle repose essentiellement sur des mécanismes simples de création, d'association et de suppression, sans logique métier avancée ni dépendances critiques.

5 - Conception du système

5.1 - Modèle de données

Le modèle de données présenté ci-dessous décrit la structure logique de la base de données du projet. Il a été conçu afin de répondre aux besoins fonctionnels identifiés dans les user stories, tout en respectant les principes de normalisation et de cohérence des données.

L'entité **Utilisateur** constitue un élément central du modèle. Elle permet de gérer l'authentification, les rôles et les statuts des utilisateurs. Un utilisateur peut être propriétaire d'un ou plusieurs projets et peut également être associé à plusieurs projets en tant que membre, via une table de liaison permettant de modéliser une relation de type *many-to-many*.

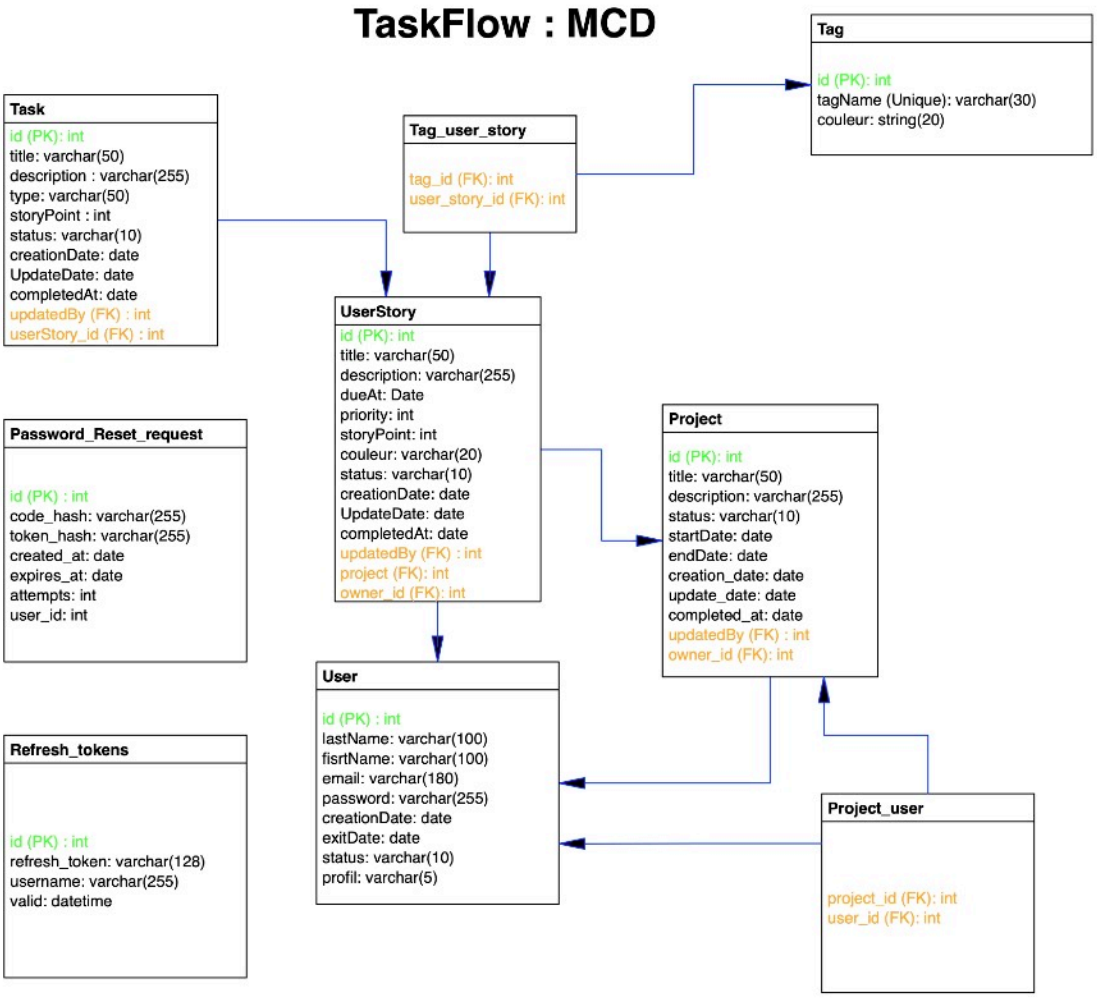
L'entité **Projet** représente un espace de travail collaboratif. Chaque projet est associé à un propriétaire unique (owner), identifié par une clé étrangère vers l'entité Utilisateur. Un projet peut contenir plusieurs user stories et regrouper plusieurs utilisateurs.

Les **User Stories** sont rattachées à un projet et permettent de structurer les besoins fonctionnels. Chaque User Story peut être associée à plusieurs tâches, ce qui permet de découper le travail de manière opérationnelle et de suivre l'avancement selon un workflow défini.

Les **Tâches** sont liées à une User Story et possèdent un créateur unique (le owner de la US). Cette relation permet de gérer les droits de modification et de suppression, conformément aux règles métier définies dans les user stories.

La gestion des **Tags** a été modélisée de manière flexible à l'aide de tables de liaison. Un tag peut être associé à plusieurs projets ou User Stories, et inversement, ce qui permet une catégorisation dynamique sans duplication de données.

Dans l'ensemble, ce modèle de données a été conçu pour être **cohérent, évolutif et conforme aux besoins fonctionnels du projet**. Il facilite la gestion des relations entre les différentes entités tout en garantissant l'intégrité des données grâce à l'utilisation de clés primaires et de clés étrangères.


TaskFlow : MCD
Tag
id (PK): int
tagName (Unique): varchar(30)
couleur: string(20)
Task
id (PK): int
title: varchar(50)
description : varchar(255)
type: varchar(50)
storyPoint : int
status: varchar(10)
creationDate: date
UpdateDate: date
completedAt: date
updatedBy (FK) : int
userStory_id (FK) : int
Tag_user_story
tag_id (FK): int
user_story_id (FK): int
UserStory
id (PK): int
title: varchar(50)
description: varchar(255)
dueAt: Date
priority: int
storyPoint: int
couleur: varchar(20)
status: varchar(10)
creationDate: date
UpdateDate: date
completedAt: date
updatedBy (FK) : int
project (FK): int
owner_id (FK): int
Password_Reset_request
id (PK) : int
code_hash: varchar(255)
token_hash: varchar(255)
created_at: date
expires_at: date
attempts: int
user_id: int
Project
id (PK): int
title: varchar(50)
description: varchar(255)
status: varchar(10)
startDate: date
endDate: date
creation_date: date
update_date: date
completed_at: date
updatedBy (FK) : int
owner_id (FK): int
User
id (PK) : int
lastName: varchar(100)
fisrtName: varchar(100)
email: varchar(180)
password: varchar(255)
creationDate: date
exitDate: date
status: varchar(10)
profil: varchar(5)
Refresh_tokens
id (PK) : int
refresh_token: varchar(128)
username: varchar(255)
valid: datetime
Project_user
project_id (FK): int
user_id (FK): int

5.2 - Diagramme de classes

Le diagramme de classes présenté ci-dessous décrit la structure logique des principales entités métier de l'application ainsi que les relations qui les unissent. Il constitue une étape clé de la conception, en faisant le lien entre l'analyse fonctionnelle exprimée sous forme de user stories et l'implémentation technique réalisée à l'aide de l'ORM Doctrine.

Ce diagramme met en évidence les entités **Utilisateur**, **Projet**, **User Story**, **Tâche** et **Tag**, qui représentent les concepts centraux du domaine métier. Les associations entre ces classes traduisent les règles de gestion définies dans le cahier des charges, notamment la gestion des rôles, des droits d'accès et de la collaboration au sein des projets.

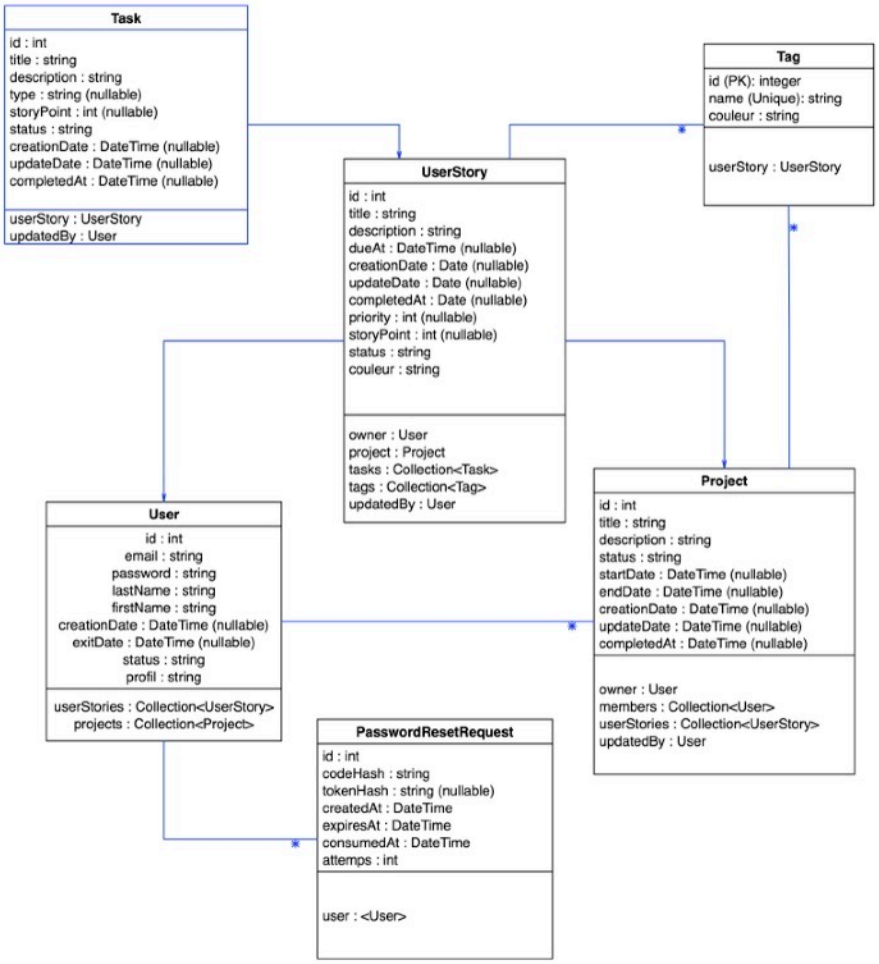
La relation entre l'utilisateur et le projet est modélisée de manière distincte afin de différencier le rôle de **propriétaire (owner)** d'un projet de celui de **membre**, reflétant ainsi les responsabilités spécifiques associées à chaque rôle. Un projet peut regrouper plusieurs utilisateurs, tout en étant rattaché à un propriétaire unique. Chaque projet contient un ensemble de user stories, qui permettent de structurer les besoins fonctionnels, elles-mêmes décomposées en tâches opérationnelles.

La gestion des tags est représentée par des relations de type *many-to-many*, offrant une grande flexibilité dans la catégorisation des projets et des user stories. Ce choix de modélisation permet d'éviter la redondance des données et de faciliter l'évolution du système.

Le diagramme de classes présenté est volontairement **simplifié et centré sur les aspects métier**. Les éléments purement techniques, tels que la gestion des tokens d'authentification ou les méthodes utilitaires, ne sont pas représentés afin de conserver une vision claire et lisible du domaine fonctionnel. Les règles métier plus complexes, notamment celles liées aux changements de statut et à la gestion des dates de complétion, sont implémentées directement au niveau des entités et détaillées dans la partie consacrée à l'implémentation.

Ainsi, ce diagramme de classes constitue une représentation cohérente et structurée du modèle de données de l'application, servant de référence pour le développement et garantissant l'alignement entre la conception et la réalisation du projet.

TaskFlow: CLASS DIAGRAM



5.3 - Diagramme des contrôleurs

Le diagramme de contrôleurs présenté ci-dessus illustre l'architecture de la couche applicative de l'API REST développée dans le cadre de ce projet. Il met en évidence la répartition des responsabilités entre les différents contrôleurs et les principales routes exposées par le système.

Chaque contrôleur est dédié à un domaine fonctionnel précis, conformément au principe de séparation des responsabilités. Cette organisation permet de garantir une meilleure lisibilité du code, une maintenance facilitée et une évolution plus maîtrisée de l'application. Les contrôleurs sont principalement structurés autour des entités métier centrales, à savoir les utilisateurs, les projets, les user stories, les tâches et les tags.

Les contrôleurs liés à l'authentification et à la sécurité, tels que **LoginController** et **RefreshTokenController**, assurent respectivement la connexion des utilisateurs et le rafraîchissement des tokens d'authentification. Ces composants permettent de sécuriser l'accès à l'API tout en offrant une expérience utilisateur fluide grâce à la gestion des sessions par token.

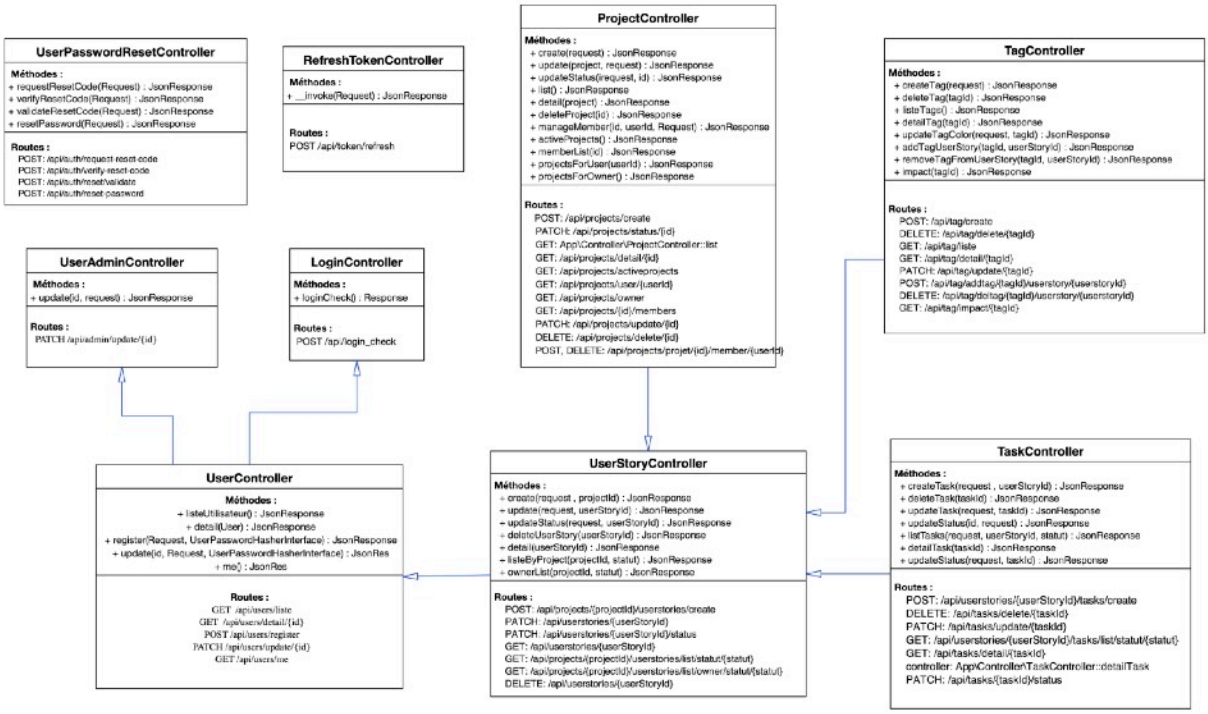
La gestion des utilisateurs est répartie entre **UserController**, responsable des opérations courantes (consultation, mise à jour du profil, inscription), et **User/Controller**, dédié aux actions administratives telles que la modification du statut ou du rôle d'un utilisateur. Cette distinction reflète la séparation des droits entre utilisateurs standards et administrateurs/managers.

Le **ProjectController** constitue un élément central de l'architecture. Il regroupe les opérations liées à la création, la consultation, la mise à jour et la suppression des projets, ainsi que la gestion des membres et des changements de statut. Les contrôleurs **UserStoryController** et **TaskController** s'inscrivent dans une hiérarchie fonctionnelle, les user stories étant rattachées à un projet et les tâches à une User Story. Cette structuration permet de refléter fidèlement l'organisation du travail définie lors de l'analyse fonctionnelle.

Enfin, le **TagController** assure la gestion transversale des tags, tant au niveau des projets que des user stories. Il permet la création, la suppression, la modification et l'association des tags, facilitant ainsi la catégorisation et le filtrage des éléments du système.

Le diagramme de contrôleurs propose volontairement une **vue synthétique** des routes et des méthodes exposées. Certaines règles de sécurité, contraintes métier ou vérifications spécifiques ne sont pas détaillées graphiquement afin de préserver la lisibilité du schéma. Ces aspects sont implémentés directement au niveau des contrôleurs et décrits dans la partie consacrée à l'implémentation technique.

Ainsi, ce diagramme offre une vision claire et structurée de l'architecture de l'API, mettant en évidence les interactions entre les différentes composantes applicatives et assurant la cohérence entre la conception et la réalisation du projet.


UserPasswordResetController
Méthodes :
+ requestResetCode(Request) : JsonResponse
+ verifyResetCode(Request) : JsonResponse
+ validateResetCode(Request) : JsonResponse
+ resetPassword(Request) : JsonResponse
Routes :
POST: /api/auth/request-reset-code
POST: /api/auth/verify-reset-code
POST: /api/auth/resetvalidate
POST: /api/auth/reset-password
RefreshTokenController
Méthodes :
+ __invoke(Request) : JsonResponse
Routes :
POST /api/token/refresh
ProjectController
Méthodes :
+ create(request) : JsonResponse
+ update(project, request) : JsonResponse
+ updateStatus(request, id) : JsonResponse
+ list() : JsonResponse
+ detail(project) : JsonResponse
+ deleteProject(id) : JsonResponse
+ manageMember(id, userId, Request) : JsonResponse
+ activeProjects() : JsonResponse
+ memberList(id) : JsonResponse
+ projectsForUser(userId) : JsonResponse
+ projectsForOwner() : JsonResponse
Routes :
POST: /api/projects/create
PATCH: /api/projects/status/{id}
GET: App\Controller\ProjectController::list
GET: /api/projects/detail/{id}
GET: /api/projects/activeprojects
GET: /api/projects/user/{userId}
GET: /api/projects/owner
GET: /api/projects/{id}/members
PATCH: /api/projects/update/{id}
DELETE: /api/projects/delete/{id}
POST, DELETE: /api/projects/projet/{id}/member/{userId}
TagController
Méthodes :
+ createTag(request) : JsonResponse
+ deleteTag(tagId) : JsonResponse
+ listeTags() : JsonResponse
+ detailTag(tagId) : JsonResponse
+ updateTagColor(request, tagId) : JsonResponse
+ addTagUserStory(tagId, userStoryId) : JsonResponse
+ removeTagFromUserStory(tagId, userStoryId) : JsonResponse
+ impact(tagId) : JsonResponse
Routes :
POST: /api/tag/create
DELETE: /api/tag/delete/{tagId}
GET: /api/tag/liste
GET: /api/tag/detail/{tagId}
PATCH: /api/tag/update/{tagId}
POST: /api/tag/addtag/{tagId}/userstory/{userstoryId}
DELETE: /api/tag/deltag/{tagId}/userstory/{userstoryId}
GET: /api/tag/impact/{tagId}
UserAdminController
Méthodes :
+ update(id, request) : JsonResponse
Routes :
PATCH /api/admin/update/{id}
LoginController
Méthodes :
+ loginCheck() : Response
Routes :
POST /ap/login_check
UserController
Méthodes :
+ listeUtilisateur() : JsonResponse
+ detail(User) : JsonResponse
+ register(Request, UserPasswordHasherInterface) : JsonResponse
+ update(id, Request, UserPasswordHasherInterface) : JsonRes
+ me() : JsonRes
Routes :
GET /api/users/liste
GET /api/users/detail/{id}
POST /api/users/register
PATCH /api/users/update/{id}
GET /api/users/me
UserStoryController
Méthodes :
+ create(request , projectId) : JsonResponse
+ update(request, userStoryId) : JsonResponse
+ updateStatus(request, userStoryId) : JsonResponse
+ deleteUserStory(userStoryId) : JsonResponse
+ detail(userStoryId) : JsonResponse
+ listeByProject(projectId, statut) : JsonResponse
+ ownerList(projectId, statut) : JsonResponse
Routes :
POST: /api/projects/{projectId}/userstories/create
PATCH: /api/userstories/{userStoryId}
PATCH: /api/userstories/{userStoryId}/status
GET: /api/userstories/{userStoryId}
GET: /api/projects/{projectId}/userstories/list/statut/{statut}
GET: /api/projects/{projectId}/userstories/list/owner/statut/{statut}
DELETE: /api/userstories/{userStoryId}
TaskController
Méthodes :
+ createTask(request , userStoryId) : JsonResponse
+ deleteTask(taskId) : JsonResponse
+ updateTask(request, taskId) : JsonResponse
+ updateStatus(id, request) : JsonResponse
+ listTasks(request, userStoryId, statut) : JsonResponse
+ detailTask(taskId) : JsonResponse
+ updateStatus(request, taskId) : JsonResponse
Routes :
POST: /api/userstories/{userStoryId}/tasks/create
DELETE: /api/tasks/delete/{taskId}
PATCH: /api/tasks/update/{taskId}
GET: /api/userstories/{userStoryId}/tasks/list/statut/{statut}
GET: /api/tasks/detail/{taskId}
controller: App\Controller\TaskController::detailTask
PATCH: /api/tasks/{taskId}/status

6 - Architecture logicielle et choix techniques

6.1 Architecture générale de l'application

L'application développée repose sur une architecture de type API REST, dont l'objectif principal est de fournir un backend indépendant, capable d'être consommé par différents clients (application web, application mobile, outils tiers).
Cette approche favorise une séparation claire des responsabilités entre la couche serveur et la couche cliente, tout en améliorant la maintenabilité et l'évolutivité du système.

Le backend est structuré selon les principes du framework Symfony, en distinguant notamment :

- les contrôleurs, chargés de l'exposition des endpoints API et de la gestion des requêtes HTTP ;
- les entités et repositories, responsables de la persistance et de l'accès aux données ;
- les formulaires, utilisés pour la validation et l'hydratation des données ;
- la couche de sécurité, assurant l'authentification et le contrôle des accès.

Cette organisation permet de garantir une architecture modulaire, lisible et conforme aux bonnes pratiques de développement backend.

6.2 Organisation des contrôleurs et des routes API

Les fonctionnalités de l'application sont exposées à travers des **contrôleurs spécialisés**, chacun correspondant à un domaine fonctionnel précis : utilisateurs, projets, user stories, tâches et tags. Chaque contrôleur regroupe l'ensemble des routes nécessaires à la gestion de son entité, selon les conventions REST (création, consultation, mise à jour, suppression).

Les routes sont structurées de manière cohérente et explicite, facilitant leur compréhension et leur utilisation.
Les verbes HTTP (GET, POST, PATCH, DELETE) sont utilisés conformément à leur sémantique, ce qui permet de respecter les principes REST et d'assurer une API prévisible et homogène.

Cette organisation favorise également la **lisibilité du code**, tout en facilitant les évolutions futures et l'ajout de nouvelles fonctionnalités.

6.3 Gestion des formulaires et validation des données

La validation des données entrantes est assurée à l'aide du composant **Form de Symfony**, utilisé ici dans un contexte d'API.
Chaque opération de création ou de mise à jour s'appuie sur un formulaire dédié, permettant de définir précisément les champs attendus, leur caractère obligatoire ou facultatif, ainsi que les règles de validation associées.

Cette approche présente plusieurs avantages :

- centralisation des règles de validation ;
- séparation entre la logique de validation et la logique métier ;
- limitation des erreurs liées à des données incohérentes ou incomplètes.

Les formulaires sont utilisés en mode **CSRF désactivé**, ce choix étant justifié par le contexte d'une API sécurisée via authentification JWT.
Ils permettent également une gestion claire des mises à jour partielles (PATCH), en conservant uniquement les champs transmis par le client.

Enfin, certains champs techniques, tels que la date de création, la date de dernière modification et l'utilisateur ayant effectué la modification, ne sont volontairement pas exposés dans les formulaires.

Ces informations sont renseignées automatiquement au niveau de la couche de persistance, via un mécanisme de listener Doctrine, garantissant une traçabilité cohérente et indépendante des données fournies par le client.

6.4 - Sécurité et gestion des accès

6.4.1 - Authentification (JWT, refresh Token)

La sécurité de l'application repose sur un système d'authentification basé sur les **JSON Web Tokens (JWT)**, mis en œuvre à l'aide du bundle LexikJWTAuthentication.
Cette solution permet une authentification **stateless**, particulièrement adaptée aux architectures API REST.

Un mécanisme de **refresh token** a également été implémenté afin d'améliorer l'expérience utilisateur tout en conservant un haut niveau de sécurité.
Les tokens sont renouvelés sans nécessiter une reconnexion systématique, tout en respectant une durée de validité limitée.

6.4.2 - Gestion des rôles et profils utilisateurs

En complément, des contrôles d'accès sont appliqués à chaque action sensible :

- vérification de l'authentification de l'utilisateur ;
- vérification du statut actif du compte ;
- contrôle des rôles et profils (utilisateur standard, manager) ;
- validation de l'accès aux ressources (projets, user stories, tâches).

Ces règles permettent de garantir que seules les actions autorisées peuvent être exécutées, tout en respectant les contraintes métier définies.

Rôles:

- Visiteur: Non authentifié
- Utilisateur (UTIL) : Utilisateur authentifié standard
- Manager (MANAGER) : Utilisateur avec droits étendus
- Owner : Propriétaire de la ressource (Projet, User Story)
- Membre: Utilisateur membre d'un projet

6.4.3 - Matrice des autorisations et règles d'accès

Cette section présente la matrice des autorisations définissant les droits d'accès aux différentes fonctionnalités de l'application en fonction du profil utilisateur et du rôle contextuel (propriétaire, membre).

La matrice est directement issue de l'implémentation des contrôleurs Symfony et garantit la cohérence entre les règles métier et la sécurité applicative.

Authentification

Action	Visiteur	Utilisateur	Manager
Connexion (JWT)	Autorisé	-	-
Rafraîchissement du Token	Refusé	Autorisé	Autorisé

Gestion des utilisateurs

Action	Utilisateur	Manager
Inscription	Autorisé	Autorisé
Lister les utilisateurs actifs	Autorisé	Autorisé
Consulter un utilisateur	Autorisé	Autorisé
Modifier son profil	Autorisé	-
Modifier un autre utilisateur	Refusé	Autorisé
Modifier statut ou rôle	Refusé	Autorisé

Gestion des projets

Action	Utilisateur	Manager	Owner	Membre
Créer un projet	Refusé	Autorisé	-	-
Lister ses projets	Autorisé	Autorisé	-	-
Consulter un projet	Refusé	Refusé	Autorisé	Autorisé
Modifier un projet	Refusé	Refusé	Autorisé	Refusé

Action	Utilisateur	Manager	Owner	Membre
Modifier le statut	Refusé	Refusé	Autorisé	Refusé
Supprimer un projet	Refusé	Refusé	Autorisé	Refusé
Gérer ses membres	Refusé	Refusé	Autorisé	Refusé

Gestion des user stories

Action	Utilisateur	Owner US	Membre projet
Créer une User Story	Refusé	-	Autorisé
Lister les User Stories	Refusé	-	Autorisé
Consulter une User Story	Refusé	-	Autorisé
Modifier une User Story	Refusé	Refusé	Autorisé
Modifier le statut	Refusé	Autorisé	Refusé
Supprimer une User Story	Refusé	Autorisé	Refusé

Gestion des tâches

Action	Utilisateur	Owner US	Membre Projet
Créer une tâche	Refusé	-	Autorisé
Lister les tâches	Refusé	-	Autorisé
Consulter une tâche	Refusé	-	Autorisé
Modifier une tâche	Refusé	Autorisé	Refusé
Modifier le statut	Refusé	Autorisé	Refusé
Supprimer une tâche	Refusé	Autorisé	Refusé

Gestion des tags

Action	Utilisateur	Manager
Créer un tag	Autorisé	Autorisé
Lister les tags	Autorisé	Autorisé
Consulter un tag	Autorisé	Autorisé

Action	Utilisateur	Manager
Modifier la couleur	Autorisé	Autorisé
Supprimer un tag	Refusé	Autorisé

Associations des tags

Action	Utilisateur	Accès au projet requis
Ajouter un tag à un projet	Autorisé	Oui
Retirer un tag à un projet	Autorisé	Oui
Ajouter un tag à une User Story	Autorisé	Oui
Retirer un tag d'un User Story	Autorisé	Oui

6.4.4 Implémentation des contrôles d’accès

Les contrôles d’accès sont implémentés directement au sein des contrôleurs Symfony.
Chaque action vérifie systématiquement :

- l’authentification de l’utilisateur
- son statut (actif ou inactif)
- son rôle global
- ses droits contextuels (propriétaire ou membre)

Cette approche garantit une cohérence stricte entre les règles métier et la sécurité applicative.

6.4.5 Traçabilité et cohérence avec les tests

La matrice des autorisations a servi de base à la définition des cas de tests fonctionnels et de sécurité. Chaque règle d'accès est couverte par au moins un scénario de test, assurant la conformité de l'application aux exigences de sécurité définies.

6.5 Règles métier et gestion des statuts

Le système intègre plusieurs règles métier structurantes, notamment à travers la gestion des statuts des entités principales : projets, user stories et tâches.
Chaque entité possède un cycle de vie clairement défini, avec des transitions de statut contrôlées.

Ces transitions sont contrôlées afin de garantir la cohérence fonctionnelle.
Par exemple, certaines modifications ne peuvent être effectuées que par le propriétaire d'une ressource, tandis que d'autres sont conditionnées au statut actuel de l'entité.

La gestion explicite de ces règles métier au niveau du backend permet :

- de renforcer la fiabilité de l'application ;
- d'éviter les incohérences de données ;
- de centraliser la logique décisionnelle.

6.6 - Intégration des méthodes agiles : notion de User Story

Bien que la gestion des user stories ne soit pas explicitement exigée dans le cahier des charges initial, ce concept a été introduit afin de se rapprocher des **méthodes agiles**, et plus particulièrement de Scrum.

La User Story constitue une unité fonctionnelle intermédiaire entre le projet et la tâche.
Elle permet de :

- structurer le travail en fonctionnalités cohérentes ;
- faciliter la priorisation ;
- améliorer la lisibilité de l'avancement du projet.

Sur le plan technique, cette approche se traduit par une modélisation claire des relations entre projets, user stories et tâches, tout en conservant une grande flexibilité.
L'intégration de ce niveau supplémentaire renforce la cohérence entre le modèle de données et les pratiques de gestion de projet agiles.

7 - Tests et validation

Afin de garantir la fiabilité, la sécurité et la conformité fonctionnelle de l'application, une **suite complète de tests automatisés** a été mise en place.
Ces tests portent principalement sur les **API REST**, cœur du système, et couvrent à la fois les aspects techniques et métier.

7.1 Stratégie de tests

La stratégie adoptée repose sur les principes suivants :

- Tests orientés cas d'usage (User Stories)
- Vérification systématique des règles de **sécurité** (authentification, autorisations)
- Validation des **workflows métier** (statuts, transitions)
- Limitation de la redondance grâce à des **outils et contextes partagés**
- Priorité donnée à la **maintenabilité** et à la lisibilité

7.2 Organisation des tests

Les tests sont organisés par domaine fonctionnel :

- Utilisateurs
- Administration
- Projets
- User Stories
- Tâches
- Tags

Une classe abstraite ApiTestCase centralise les comportements communs :

- Authentification JWT
- Création de contextes métier complets
- Envoi de requêtes API authentifiées

Cette approche permet de réduire la duplication de code et de garantir une cohérence globale.

7.3 Couverture fonctionnelle

Les tests couvrent notamment :

- Les cas d'échec (401, 403, 404, 400)
- Les validations de formulaire
- Les règles d'accès aux ressources
- Les effets métier en base de données
- Les transitions de statut des entités principales

Chaque fonctionnalité critique dispose ainsi d'un ensemble de tests assurant son bon fonctionnement dans des conditions réelles d'utilisation.

7.4 - Detail des tests

7.4.1 - Utilisateurs

User stories couvertes (US1, US2, US3, US5, US6, US7)
Fichier de tests: UsersTest.php

Liste des utilisateurs (US7)

Route: GET /api/users/liste		
Cas testé	**Code attendu**	**Test**
Utilisateur authentifié	200 OK	testGetUsersWithAuthSuccessfully
Non authentifié	401 Unauthorized	testEchecGetUserFail

Détail d'un utilisateur (US6)

Route: GET /api/users/detail/{id}		
Cas testé	**Code attendu**	**Test**
Non authentifié	401 Unauthorized	testEchecGetIdUserFail
Utilisateur inexistant	404 Not Found	testGetUserDetail
Utilisateur authentifié	200 OK	testGetUserDetail
Sérialisation correcte (`user:detail`)	200 OK	testGetUserDetail

Création d'un utilisateur (US1)

Route: POST /api/users/register		
Cas testé	**Code attendu**	**Test**
Payload valide	201 Created	testCreateUserSuccessfully
Email déjà utilisé	400 Bad Request	testCreateUserFail
Mot de passe manquant / invalide	400 Bad Request	testCreateUserFail
Email invalide	400 Bad Request	testCreateUser2Fail

Contraintes du mot de passe (US1)

Route: PATCH /api/users/update/{id}		
Cas testé	**Code attendu**	**Test**
< 8 caractères	400 Bad Request	testCreateWrongEmailFail
Sans majuscule	400 Bad Request	testCreateWrongEmailFail
Sans chiffre	400 Bad Request	testCreateWrongEmailFail
Sans caractère spécial	400 Bad Request	testCreateWrongEmailFail

Mise à jour d'un utilisateur (US3)

Route: PATCH /api/users/update/{id}		
Cas testé	**Code attendu**	**Test**
Non authentifié	401 Unauthorized	testUpdateUserNameFail
UTIL modifie autre user	403 Forbidden	testUpdateUserNameFail
UTIL modifie son profil	200 OK	testUpdateUserNameSuccessfully
Manager modifie profil	200 OK	testUpdateUserNameAdminSuccess

Connexion utilisateur (US2)

Route : POST /api/login_check		
Cas testé	**Code attendu**	**Test**
Identifiants incorrects	401 Unauthorized	testLoginFail
Utilisateur INACTIVE	403 Forbidden	testLoginInactiveFail
Identifiants valides	200 OK	testLoginSuccessfully
Token/session retourné	200 OK	testLoginSuccessfully

Refresh Token / Session (US5)

Route: POST /api/token/refresh		
Cas testé	**Code attendu**	**Test**
Refresh valide	200 OK	testRefreshTokenSuccessfully

7.4.2 Administrateurs / Managers

User stories couvertes (US4)
Fichier de tests: UserAdminTest.php

Mise à jour d'un utilisateur par un manager (US4)

Route: PATCH /api/admin/update/{id}		
Cas testé	**Code attendu**	**Test**
Non authentifié	401 Unauthorized	testUpdateUserNonAuthFail
Non manager	403 Forbidden	testUpdateUserNonAdminFail
Utilisateur inexistant	404 Not Found	testUpdateAdminUserInexistantFail
Manager actif	200 OK	testUpdateAdminSuccess
Modification profil/status	200 OK	testUpdateAdminSuccess

7.4.3 - Projets

User stories couvertes (US8, US9, US10, US11, US12)
Fichier de tests: ProjectTest.php

Création d'un projet (US8)

Route: POST /api/projects/create		
Cas testé	**Code attendu**	**Test**
Non authentifié	401 Unauthorized	testProjectCreationUnauthenticatedFail
Non manager	403 Forbidden	testProjectCreationUtilFail
Formulaire invalide	400 Bad Request	testProjectCreationInvalidFormFail
Manager actif	201 Created	testProjectCreationMgrSuccess
Owner = connecté	201 Created	testProjectCreationMgrSuccess

Mise à jour d'un projet (US8)

Route: PATCH /api/projects/update/{id}		
Cas testé	**Code attendu**	**Test**
Non authentifié	401 Unauthorized	testProjectUpdateUnauthenticatedFail
Non manager	403 Forbidden	testProjectUpdateByUtilFail
Non owner	403 Forbidden	testProjectUpdateByUtilFail
Formulaire invalide	400 Bad Request	testProjectUpdateInvalidFormFail
Owner manager actif	200 OK	testProjectUpdateByOwnerSuccess
PATCH partiel	200 OK	testProjectUpdateByOwnerSuccess

Suppression d'un projet (US9)

Route: DELETE /api/projects/delete/{id}		
Cas testé	**Code attendu**	**Test**
Non authentifié	401 Unauthorized	testProjectDeleteUnauthenticatedFail
Non owner	403 Forbidden	testProjectDeleteByMgrFail
Projet inexistant	404 Not Found	testProjectDeleteInexistantFail
Owner	204 No Content	testProjectDeleteByMgrSucces

Ajout d'un membre sur un projet (US10)

Route: POST /api/projects/projet/{id}/member/{userId}		
Cas testé	**Code attendu**	**Test**
Non authentifié	401 Unauthorized	testProjectAddMemberUnauthenticated
Pas accès	403 Forbidden	testProjectAddMemberWithoutAccessFa
Projet inexistant	404 Not Found	testProjectAddMemberProjectNotFound
User inexistant	404 Not Found	testProjectAddMemberUserNotFoundFai
Ajout OK	201 Created	testProjectAddMemberByOwnerSuccess
Pas duplication	200 OK	testProjectAddMemberDoesNotDuplicat

Retrait d'un membre sur un projet (US11)

Route: DELETE /api/projects/projet/{id}/member/{userId}		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testProjectRemoveMemberUnauthenticatedFail
Utilisateur sans accès au projet	403 Forbidden	testProjectRemoveMemberWithoutAccessFail
Utilisateur non membre du projet	204 No Content	testProjectRemoveNonMemberReturns204
Suppression réussie	204 No Content	testProjectRemoveMemberByOwnerSuccess

Liste des projets (US12)

Route: GET /api/projects/liste		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testProjectListUnauthenticatedFail
Owner consulte la liste	200 OK	testProjectListOnlyAccessibleProjects
Membre consulte la liste	200 OK	testProjectListOnlyAccessibleProjects
Projets sans accès invisibles	200 OK	testProjectListOnlyAccessibleProjects

Détail d'un projet (US12)

Route: GET /api/projects/detail/{id}		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testProjectDetailUnauthenticatedFail
Utilisateur sans accès au projet	403 Forbidden	testProjectDetailNoAccessFail
Projet inexistant	404 Not Found	testProjectDetailNotFoundFail
Owner ou membre du projet	200 OK	testProjectDetailSuccessForOwnerAndMember

Liste des projets en cours (US12)

Route: GET /api/projects/activeprojects		
Cas testé	**Code attendu**	**Test**
Non authentifié	401 Unauthorized	testActiveProjectsUnauthenticatedFail
Retourne uniquement les projets actifs	200 OK	testActiveProjectsReturnsOnlyActiveAndAccessibleProjects
Retourne uniquement les projets accessibles	200 OK	testActiveProjectsReturnsOnlyActiveAndAccessibleProjects

Workflow

Route : PATCH /api/projects/status/{id}		
Transition	**Code attendu**	**Test**
NS -> ENC	200	testUpdateProjectStatusNsToEncSuccess
NS -> TER	200	testUpdateProjectStatusNsToTerSuccess
ENC -> TER	200	testUpdateProjectStatusEncToTerSuccess
TER -> ENC	200	testUpdateProjectStatusTerToEncSuccess
TER -> NS	200	testUpdateProjectStatusTerToNsSuccess
Pas accès	403	testUpdateProjectStatusForbidden
Projet inexistant	404	testUpdateProjectStatusNotFound

7.4.4 - User stories

User stories couvertes (US13, US14, US15, US16, US17, US18)
Fichier de tests: UserStoryTest.php

Création d'un User Story (US13)

Route: POST /api/projects/{projectId}/userstories/create		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testUSCreationUnauthenticatedFail
Projet inexistant	404 Not Found	testCreateUsProjectNotFound
Utilisateur sans accès au projet	403 Forbidden	testCreateUsProjectForbidden
Formulaire invalide	400 Bad Request	testCreateUsInvalidForm
Création par owner	201 Created	testCreateUsSuccess
Status par défaut = NS	201 Created	testCreateUsSuccess
User Story liée au bon projet et owner	201 Created	testCreateUsSuccess

Suppression d'un User Story (US14)

Route: DELETE /api/userstories/{userStoryId}		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testDeleteUserStoryUnauthenticatedFail
User Story inexistante	404 Not Found	testDeleteUserStoryNotFound
Utilisateur sans accès au projet	403 Forbidden	testDeleteUserStoryForbidden
Suppression par owner	204 No Content	testDeleteUserStorySuccess

Liste des user stories (US16)

Route: GET /api/projects/{projectId}/userstories/list/statut/{statut}		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testUSListUnauthenticatedFail
Projet inexistant	404 Not Found	testListeUsProjectNotExist
Utilisateur sans accès au projet	403 Forbidden	testListeUserStoryForbidden
Owner du projet	200 OK	testListeUserStoryByOwnerSuccess
Membre du projet	200 OK	testListeUserStoryByMemberSuccess

Détail d'un User Story (US16)

Route: GET /api/userstories/{userStoryId}		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testUSDetailUnauthenticatedFail
User Story inexistante	404 Not Found	testDetailUserStoryNotFound
Utilisateur sans accès au projet	403 Forbidden	testDetailUserStoryForbidden
Accès autorisé (owner / membre)	200 OK	testDetailUserStorySuccess

Mise à jour d'une User Story (US15)

Route: PATCH /api/userstories/{userStoryId}		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testUSUpdateUnauthenticatedFail
Utilisateur sans accès au projet	403 Forbidden	testUpdateUserStoryForbidden
User Story inexistante	404 Not Found	testUpdateUserStoryNotFound
Formulaire invalide	400 Bad Request	testUpdateUserStoryInvalid
Mise à jour partielle (PATCH)	200 OK	testUpdateUserStorySuccess

Workflow (US17 / US18)

Route : PATCH /api/userstories/{userStoryId}/status		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testUpdateUserStoryStatusUnauthenticatedFail
Transition NS -> ENC	200 OK	testUpdateUserStoryStatusNsToEncSuccess
Transition NS -> TER	200 OK	testUpdateUserStoryStatusNsToTerSuccess
Transition ENC -> TER	200 OK	testUpdateUserStoryStatusEncToTerSuccess
Transition TER -> ENC	200 OK	testUpdateUserStoryStatusTerToEncSuccess
Transition TER -> NS	200 OK	testUpdateUserStoryStatusTerToNsSuccess
Utilisateur sans accès au projet	403 Forbidden	testUpdateUserStoryStatusForbidden
User Story inexistante	404 Not Found	testUpdateUserStoryStatusNotFound

7.4.5 - Tâches: TaskTest.php

User stories couvertes (US19, US20, US21, US22)
Fichier de tests: TaskTest.php

Création d'une tâche (US19)

Route: POST /api/userstories/{userStoryId}/tasks/create		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testTaskCreationUnauthenticatedFail
User Story inexistante	404 Not Found	testCreateTaskNotFound
Utilisateur sans accès au projet	403 Forbidden	testCreateTaskForbidden
Formulaire invalide	400 Bad Request	testCreateTaskInvalid
Création valide (owner)	201 Created	testCreateTaskSuccess
Status par défaut = NS	201 Created	testCreateTaskSuccess
Tâche liée à la bonne User Story	201 Created	testCreateTaskSuccess

Suppression d'une tâche (US22)

Route: DELETE /api/tasks/delete/{taskId}		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testTaskDeleteUnauthenticatedFail
Tâche inexistante	404 Not Found	testDeleteTaskNotFound
Utilisateur ≠ owner de la User Story	403 Forbidden	testDeleteTaskForbidden
Suppression valide (owner)	204 No Content	testDeleteSuccess
Suppression effective en base	204 No Content	testDeleteSuccess

Mise à jour d'une tâche (US20)

Route: PATCH /api/tasks/update/{id}		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testTaskUpdateUnauthenticatedFail
Tâche inexistante	404 Not Found	testUpdateTaskNotFound
Utilisateur ≠ owner de la User Story	403 Forbidden	testUpdateTaskForbidden
Formulaire invalide	400 Bad Request	testUpdateTaskInvalid
Mise à jour partielle (PATCH)	200 OK	testUpdateTaskSuccess

Liste des tâches (US19)

Route: GET /api/tasks/update/{taskId}		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testTaskListeUnauthenticatedFail
User Story inexistante	404 Not Found	testListeTaskNotFound
Utilisateur sans accès au projet	403 Forbidden	testListeTaskForbidden
Owner de la User Story	200 OK	testListeTaskOwnerSuccess
Membre du projet	200 OK	testListeTaskMemberSuccess
Tâches d'un autre projet invisibles	200 OK	testListeTaskOtherProjectInvisible

Détail d'une tâche (US19)

Route: GET /api/tasks/detail/{taskId}		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testTaskDetailUnauthenticatedFail
Tâche inexistante	404 Not Found	testDetailTaskNotFound
Utilisateur sans accès au projet	403 Forbidden	testLDetailTaskForbidden
Accès autorisé (membre / owner)	200 OK	testDetailTaskMemberSuccess

Workflow de statut d'une tâche (US20 & US21)

Route : PATCH /api/tasks/{taskId}/status		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testUpdateTaskStatusUnauthenticatedFail
Transition NS -> ENC	200 OK	testUpdateTaskStatusNsToEncSuccess
Transition NS -> TER	200 OK	testUpdateTaskStatusNsToTerSuccess
Transition ENC -> TER	200 OK	testUpdateTaskStatusEncToTerSuccess
Transition TER -> ENC (completedAt vidé)	200 OK	testUpdateTaskStatusTerToEncClearsCompletedAt
Transition TER -> NS (completedAt vidé)	200 OK	testUpdateTaskStatusTerToNSClearsCompletedAt
Utilisateur sans accès au projet	403 Forbidden	testUpdateTaskStatusForbidden
Tâche inexistante	404 Not Found	testUpdateTaskStatusNotFound

7.4.6 - Tags: TagTest.php

User stories couvertes (US23, US24, US25)
Fichier de tests: TagTest.php

Création d'un tag (US23)

Route: POST /api/tag/create		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testTagCreationUnauthenticatedFail
Formulaire invalide	400 Bad Request	testTagCreationInvalidFormFail
Tag déjà existant	400 Bad Request	testTagCreationAlreadyExistsFail
Création valide	201 Created	testTagCreationSuccess
Normalisation du tagName	201 Created	testTagNameIsNormalized

Suppression d'un tag (US25)

Route: DELETE /api/tag/delete/{id}		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testDeleteTagUnauthenticatedFail
Utilisateur non manager	403 Forbidden	testDeleteTagUnauthenticatedFail
Tag inexistant	404 Not Found	testDeleteTagNotFound
Suppression valide (manager & tag libre)	204 No Content	testDeleteTagSuccess

Liste des tags (US23)

Route: GET /api/tag/liste		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testListTagsUnauthenticatedFail
Liste retournée (tags triés par nom)	200 OK	testListTagsSorted

Détail d'un tag (US23)

Route: GET /api/tag/detail/{tagId}		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testTagImpactUnauthenticatedFail
Tag inexistant	404 Not Found	testTagImpactNotFound
Accès valide	200 OK	testTagImpactWithoutAccess
Owner consulte le détail / impact	200 OK	testTagImpactOwnerSeesProjects

Mise à jour d'un tag (US23)

Route: PATCH /api/tag/update/{tagId}		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testUpdateTagUnauthenticatedFail
Couleur invalide	400 Bad Request	testUpdateTagInvalidColorFail
Tag inexistant	404 Not Found	testUpdateTagNotFound
Mise à jour valide (couleur correcte)	200 OK	testUpdateTagSuccess

Ajout d'un tag à une User Story (US24)

Route: POST /api/tag/addtag/{tagId}/userstory/{userstoryId}		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testAddTagToUserStoryUnauthenticate
Utilisateur sans accès au projet	403 Forbidden	testAddTagToUserStoryForbidden
Tag déjà associé à la User Story	409 Conflict	testAddTagToUserStoryAlreadyLinked
Association valide	200 OK	testAddTagToUserStorySuccess

Retrait d'un tag d'une User Story (US25)

Route: DELETE /api/tag/deltag/{tagId}/userstory/{userstoryId}		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testRemoveTagFromUserStoryUnauthenticatedFail
Utilisateur sans accès au projet	403 Forbidden	testRemoveTagFromUserStoryForbidde
Tag non associé à la User Story	404 Not Found	testRemoveTagFromUserStoryNotLinke
Suppression valide	200 OK	testRemoveTagFromUserStorySuccess

Normalisation et unicité du tag (US23)

Route : POST /api/tag/create		
Cas testé	**Code attendu**	**Test**
Création "Backend" puis "backend"	400 Bad Request	testTagNameUniquenessCaseInsensitive
Couleur invalide	400 Bad Request	testTagCreationInvalidColorFail
Normalisation en minuscule	201 Created	testTagNameNormalizedToLowercase

Suppression d'un tag et propagation (US25)

Route : DELETE /api/tag/delete/{tagId}		
Cas testé	**Code attendu**	**Test**
Suppression d'un tag encore lié	409 Conflict	testDeleteLinkedTagFail
Suppression d'un tag libre	204 No Content	testDeleteFreeTagSuccess

Recherche des user stories et projets portant un tag donné (US26)

Route : GET /api/tag/impact/{tagId}		
Cas testé	**Code attendu**	**Test**
Utilisateur non authentifié	401 Unauthorized	testTagImpactNoAccessReturnsEmpty
Owner consulte l'impact du tag	200 OK	testTagImpactOwnerProjects
Membre consulte l'impact du tag	200 OK	testTagImpactMemberSeesProjects

7.5 - Résultat des tests

```
Time: 03:01.017, Memory: 85.00 MB

OK (155 tests, 402 assertions)

Faker seed used: 836144
```

8 – Application iOS (Swift)

8.1 Objectif de l'application mobile

L'application iOS constitue le client mobile natif du système TaskFlow.

Elle permet aux utilisateurs d'interagir avec l'API REST Symfony afin de :

- S'authentifier
- Administrer les utilisateurs (profil Manager)
- Consulter et gérer leurs projets
- Gérer les user stories
- Gérer les tâches

L'objectif est d'offrir une expérience fluide, réactive et sécurisée.

8.2 - Architecture de l'application

L'application repose sur une architecture **MVVM (Model – View – ViewModel)**.

8.2.1 - Model

Les modèles représentent les données issues du backend :

- ProfileResponse
- ProjectResponse
- StoryResponse
- TaskResponse
- TagResponse

Ils implémentent Codable pour permettre :

- Décodage JSON
- Sérialisation réseau
- Mapping direct API : Swift

Exemple :

```
struct ProjectResponse: Codable, Identifiable {
    let id: Int
    let title: String
    let description: String
    let status: ProjectStatus
    let owner: ProfileLiteResponse
    let membersCount: Int
}
```

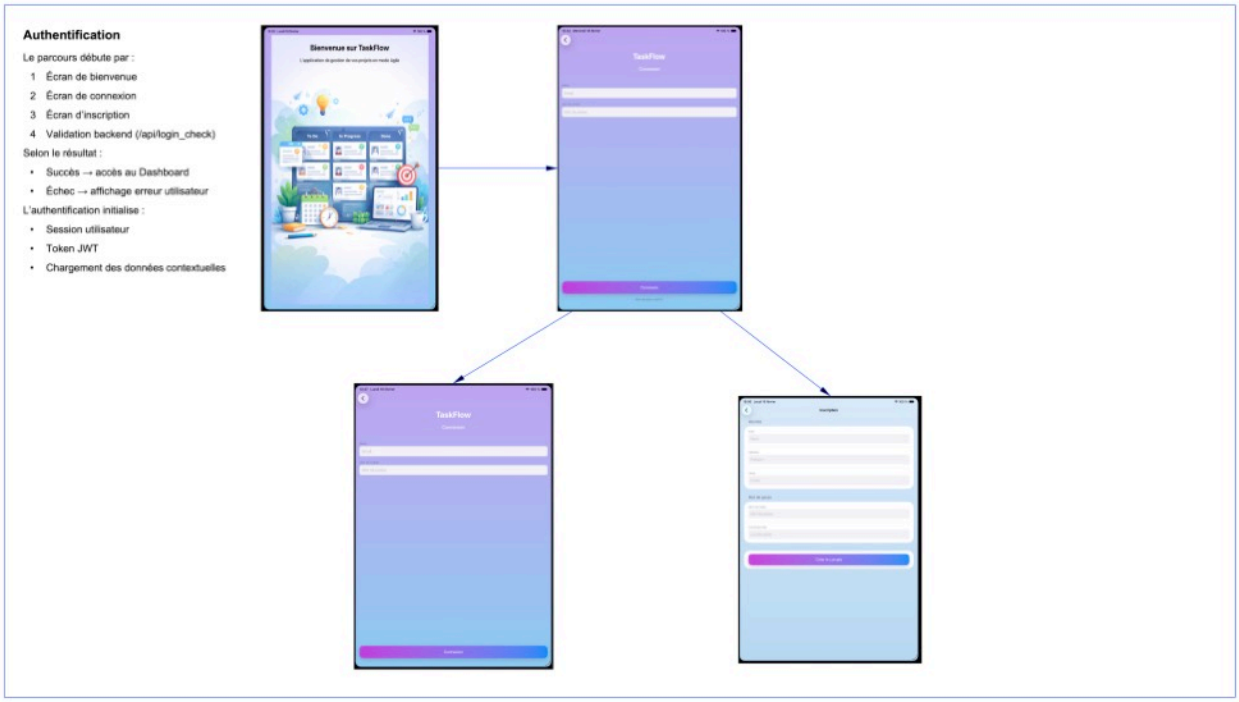
8.2.2 - Cinématique

La cinématique de TaskFlow décrit le parcours utilisateur depuis l'authentification jusqu'aux différents modules fonctionnels (Backlog, Projets, Tâches, Utilisateurs, Profil).

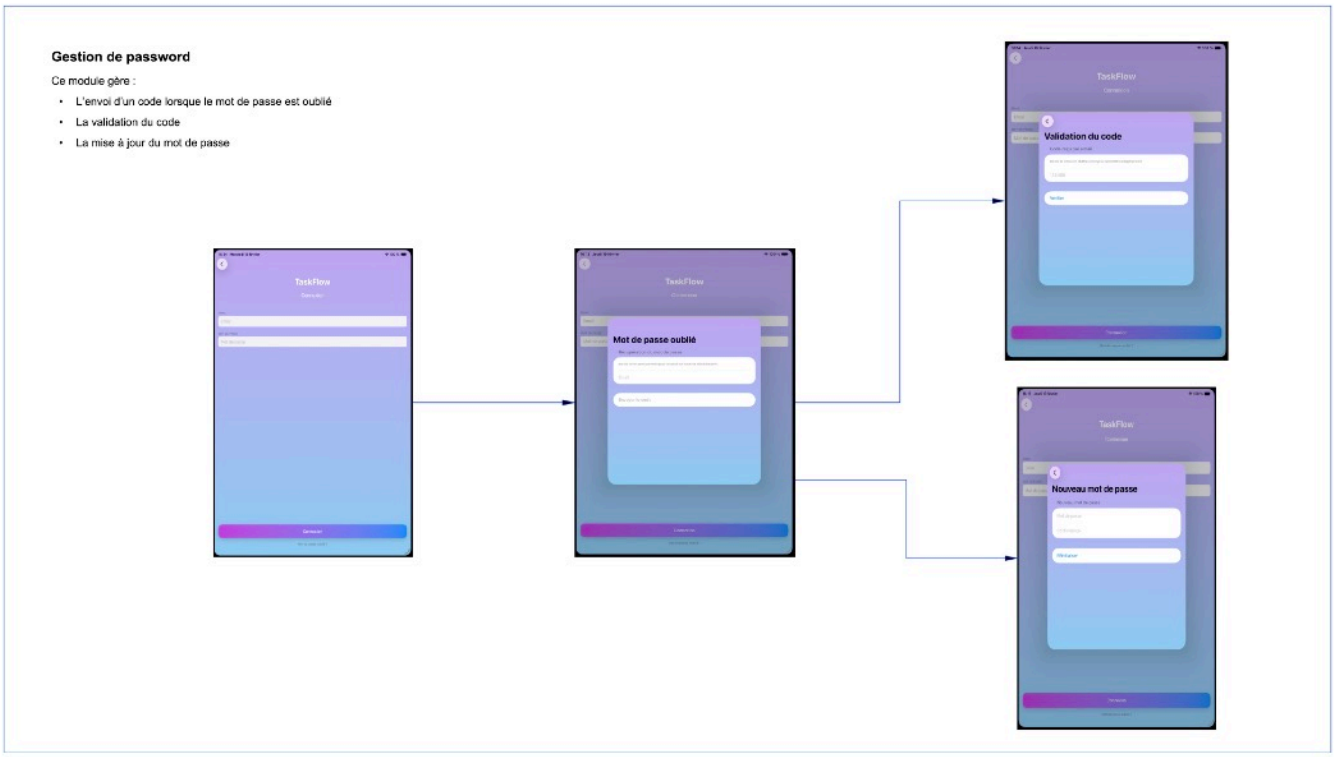
L'application s'appuie sur NavigationStack pour les flux principaux et sur des présentations modales pour les actions d'édition.

Chaque interaction métier déclenche une synchronisation avec le backend garantissant la cohérence des données et de l'interface.

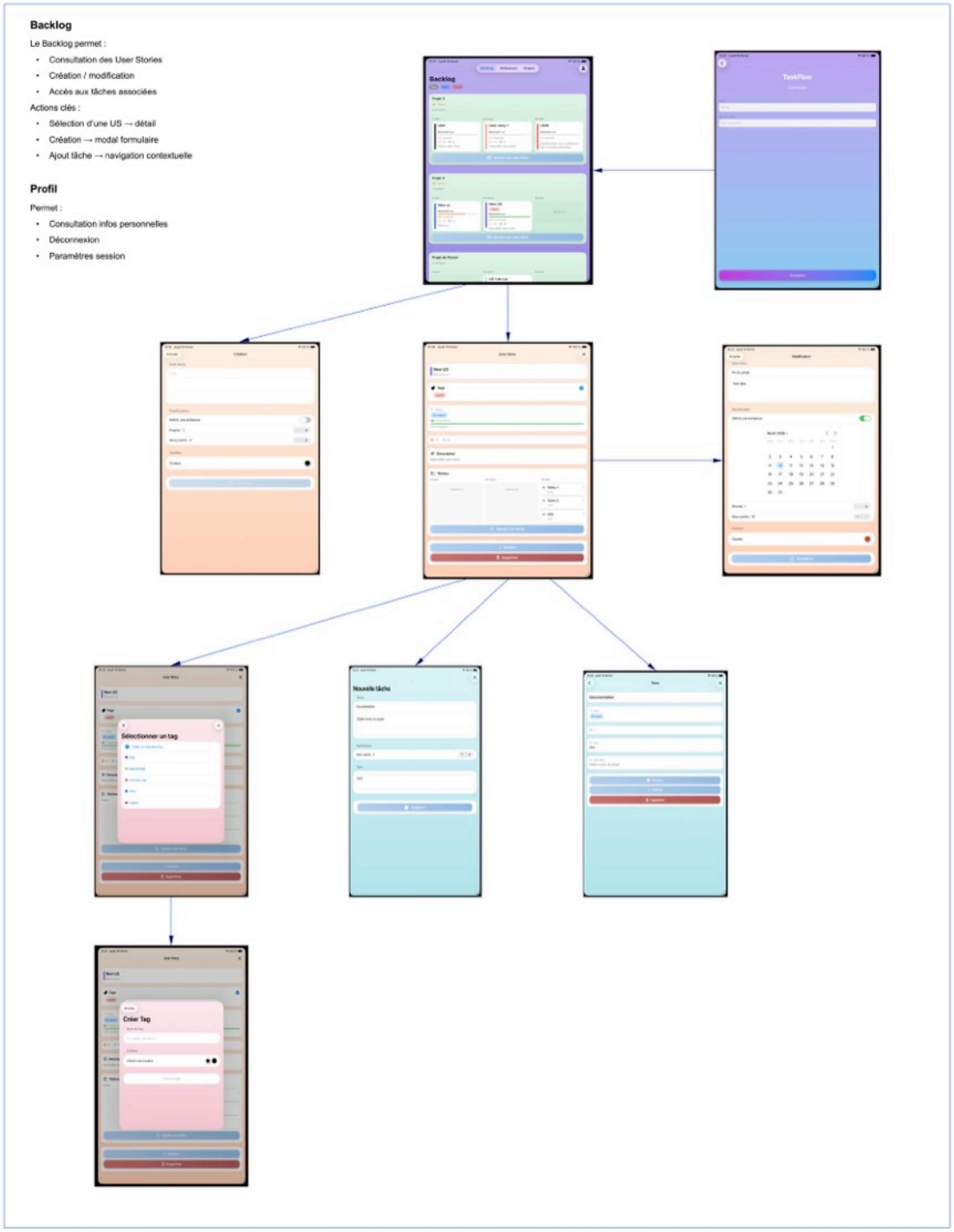
8.2.2.1 - Connexion



8.2.2.2 - Mots de passe oublié

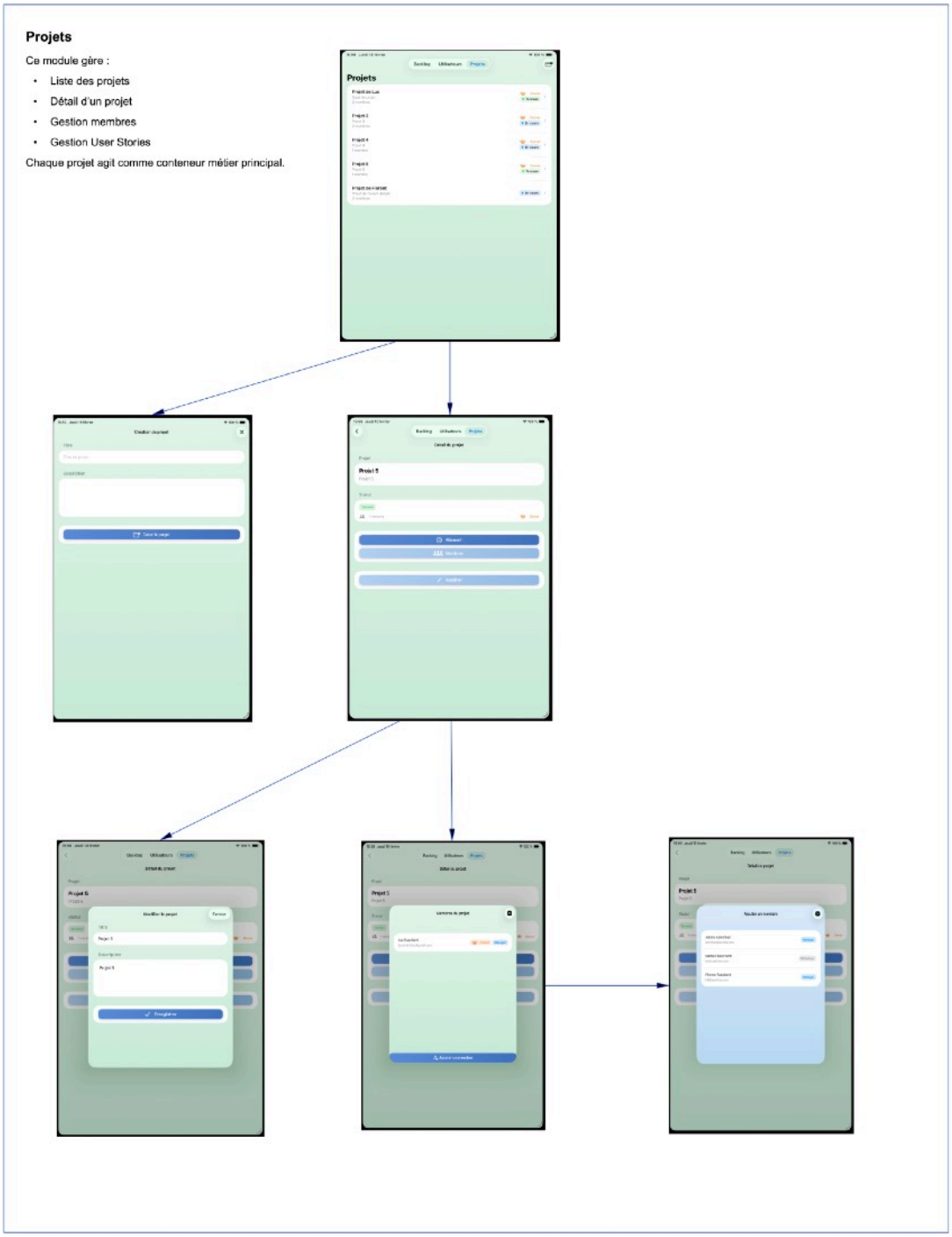


8.2.2.3 - Backlog



8.2.2.4 - Utilisateurs

8.2.2.5 - Projets



8.2.3 - Views

Les vues SwiftUI sont responsables :

- De l’affichage
- Des interactions utilisateur
- De la navigation

Elles observent les ViewModels via :

- @StateObject
- @ObservedObject
- @EnvironmentObject

8.2.4 - ViewModel

Les ViewModels encapsulent :

- Logique métier
- Appels API
- Gestion des états UI

Exemple :

```
@Published var isLoading = false
@Published var errorMessage: String?
```

Ils assurent :

- Séparation UI / logique
- Testabilité
- Maintenabilité

8.3 - Gestion de la navigation

La navigation globale repose sur :

- AppState
- Enum Flow

```
enum Flow {
   case welcome
   case loginHome
   case loginForm
   case main
}
```

Cette approche permet :

- Navigation centralisée
- Contrôle du cycle d’authentification
- Transitions propres entre écrans

8.4 - Communication avec le Backend

8.4.1 - APIClient

APIClient centralise :

- Construction des requêtes HTTP
- Injection du JWT
- Encodage JSON
- Décodage des réponses
- Gestion des erreurs HTTP

Responsabilités :

- Réduction de la duplication
- Standardisation des appels API

8.4.2 - Services réseau

Chaque domaine métier possède son service :

- LoginService
- ProfileService
- ProjectService
- StoriesService
- TaskService
- TagsService
- UsersListService

Avantages :

- Séparation claire
- Code modulaire
- Facilité d’évolution

8.4.3 - Exemple d’appel API

```
func listProjects() async throws -> [ProjectResponse] {
    let url = AppConfig.baseURL.appendingPathComponent("/api/projects/liste")
    return try await APIClient.shared.request(url: url)
}
```

8.5 - Sécurité et authentification

8.5.1 - JWT

Authentification via :

- /api/login_check
- Access Token
- Refresh Token

8.5.2 - SessionManager

Responsable de :

- Stockage Access Token
- Stockage sécurisé Refresh Token (Keychain)
- Déconnexion
- Restauration session

8.5.3 - Biométrie (Face ID / Touch ID)

Utilisation de :

- LocalAuthentication

Objectif :

- Reconnexion sécurisée
- UX améliorée
- Protection du Refresh Token

8.6 – Choix techniques

Le développement de l'application iOS *TaskFlow* s'est appuyé sur plusieurs choix techniques visant à garantir la maintenabilité du code, la cohérence de l'interface utilisateur et l'adaptabilité aux différents environnements d'exécution.

8.6.1 – Architecture MVVM

L'application adopte le pattern **MVVM (Model – View – ViewModel)**.

Cette architecture offre plusieurs avantages :

- séparation claire des responsabilités
- découplage entre logique métier et interface graphique
- testabilité accrue
- meilleure évolutivité

Les ViewModels gèrent :

- les appels API
- la logique métier
- la transformation des données pour l'affichage

Les Views restent quant à elles concentrées sur la présentation et l'interaction utilisateur.

Chaque ViewModel expose :

- isLoading
- errorMessage
- Données observables

Permet :

- Feedback utilisateur
- Gestion erreurs réseau
- États vides

8.6.2 – Gestion de la navigation

La navigation repose sur **NavigationStack**.

Cette solution permet :

- une navigation hiérarchique claire
- la gestion d'écrans dynamiques
- une compatibilité avec les destinations conditionnelles (navigationDestination)
- une meilleure cohérence avec SwiftUI moderne

Les transitions modales utilisent :

- .sheet() pour les interactions légères
- .fullScreenCover() pour les écrans immersifs (édition, création)

8.6.3 – Programmation asynchrone

Les échanges avec le backend exploitent **Swift Concurrency** (async/await).

Avantages :

- syntaxe simplifiée
- code plus lisible que les closures imbriquées
- meilleure gestion des erreurs (do / try / catch)
- fluidité de l’interface utilisateur

Les opérations réseau s’exécutent hors du thread principal afin d’éviter tout blocage visuel.

8.6.4 – Adaptation multi-appareils

L'interface a été conçue selon une approche **responsive** afin de s'adapter aux différents formats :

- iPhone (compact)
- iPad (regular)
- macOS (mode iPad)

Cette adaptation repose sur :

- horizontalSizeClass
- des layouts flexibles (maxWidth: .infinity)
- des constantes UI centralisées (UIConstants)

Des dimensions dynamiques sont appliquées aux composants clés (cartes, colonnes, sections), garantissant une expérience utilisateur cohérente sur toutes les tailles d'écran.

8.6.5 – Centralisation des constantes UI

Afin d'assurer une homogénéité visuelle et faciliter les ajustements, les dimensions principales sont définies dans une structure dédiée :

```
enum UIConstants {
    static let cardHeight: CGFloat = 160
    static let cardWidthCompact: CGFloat = 180
    static let cardWidthRegular: CGFloat = 240
    static let cardWidthMac: CGFloat = 320
}
```

Cette centralisation permet :

- une modification rapide du design
- une cohérence entre User Stories et Tasks
- une meilleure maintenabilité

8.6.6 – Gestion des thèmes clair / sombre

L’application prend en charge les modes :

- Light
- Dark

Cette compatibilité est assurée par :

- l’usage de couleurs dynamiques (Color.primary, Color.secondary)
- l’utilisation de systemBackground
- des gradients adaptatifs (BackgroundView)
- l’évitement des couleurs fixes inadaptées

Ce choix améliore :

- la lisibilité
- le confort visuel
- la conformité aux standards Apple

8.6.7 – Expérience utilisateur et interactions

Plusieurs mécanismes SwiftUI ont été intégrés :

- **Drag & Drop** pour déplacer Stories et Tasks
- **Animations légères** pour renforcer la perception des actions
- **Feedback visuel** lors des zones de dépôt
- Mise à jour optimiste de l'UI

Ces éléments contribuent à une interaction fluide et intuitive.

8.7 - Liens Backend - iOS

Mapping JSON / Swift

Les structures Swift reflètent directement :

- Entités backend
- DTO Symfony
- Réponses API

Exemple :

Backend -> Project
Swift -> ProjectResponse

Endpoints consommés

Fonction	Endpoint
Login	/api/login_check
Profil	/api/users/me
Projets	/api/projects/liste
Stories	/api/projects/{id}/userstories/...
Tâches	/api/tasks/...
Tags	/api/tag/...

8.8 – Description des écrans

8.8.1 – Authentification

8.8.1.1 – Page de logo

View :

WelcomeView

Rôle

Écran d'accueil "splash" affiché au lancement de l'app avant d'entrer dans le flow de connexion.

Description

Affiche un fond général, un titre, un sous-titre et une illustration ("Accueil").
Déclenche automatiquement une transition vers l'écran de connexion après un délai.

Fichiers Swift associés

Élément	Type	Description
BackgroundView.swift	UI Component	Gestion du fond visuel et du thème écran
TitreView.swift	UI Component	Affichage du titre principal
SousTitreView.swift	UI Component	Affichage du sous-titre
AppState.swift	State	Gestion du flow global et de la navigation
Asset image : Accueil	Ressource	Illustration affichée sur l'écran Welcome

Navigation

Type	Détail
Entrée	Affichée lorsque AppState.flow = welcome
Sortie	ConnexionView (transition automatique ~2,5 s)
Conditions	Navigation automatique
Dépendances	AppState (@EnvironmentObject)

API

Aucune.

Règles de gestion

ID	Règle
WEV01	La vue est visible uniquement dans le flow **non authentifié** (étape *welcome*).
WEV02	Une transition automatique vers l'écran **loginHome** est déclenchée après un délai (effet splash).

8.8.1.2 – Page d’accueil

View :

ConnexionView

Rôle

Écran d’orientation du flow non authentifié permettant à l’utilisateur de choisir entre connexion et inscription.

Description

Écran d’accueil servant de hub de navigation.
Affiche un fond, un titre/sous-titre et deux actions : **Connexion** et **Inscription**.
Aucune logique métier.

Fichiers Swift associés

Élément	Type	Description
LoginView.swift	Vue	Écran d’authentification utilisateur
RegisterView.swift	Vue	Création / édition de compte
BackgroundView.swift	UI Component	Gestion du fond et du thème visuel
TitreView.swift	UI Component	Affichage du titre principal
SousTitreView.swift	UI Component	Affichage du sous-titre
BoutonView.swift	UI Component	Boutons d’action stylisés
SessionViewModel.swift	ViewModel	Gestion de la session utilisateur globale

Navigation

Type	Détail
Entrée	Affichée dans le flow non authentifié (loginHome)
Sorties	LoginView (bouton Connexion) RegisterView(mode: .create) (bouton Inscription)
Conditions	Navigation déclenchée par action utilisateur
Dépendances	SessionViewModel (@EnvironmentObject)

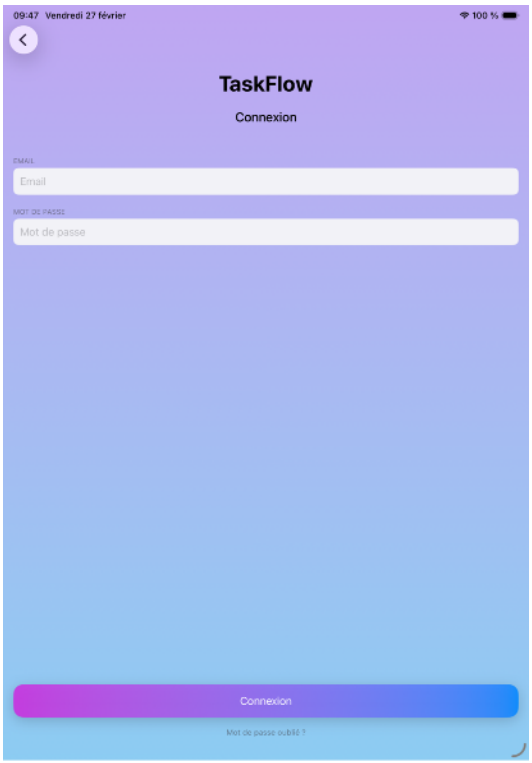
API

Aucune.

Règles de gestion

ID	Règle
COV01	La vue est visible uniquement lorsque l'utilisateur est **non authentifié**.
COV02	La connexion redirige l'utilisateur vers LoginView
COV03	L'inscription redirige l'utilisateurs vers RegisterView

8.8.1.3 – Page de connexion



View :

LoginView

Rôle

Permet à l'utilisateur de s'authentifier (email / mot de passe) et de restaurer une session via biométrie si une session valide est stockée.

Description

Formulaire de connexion avec email, mot de passe et action *Connexion*.
Affiche un message d'erreur si nécessaire et un lien *Mot de passe oublié ?*.
Tentative biométrique automatique si session existante.

Fichiers Swift associés

Élément	Type	Description
LoginViewModel.swift	ViewModel	Gestion des états UI et logique de connexion
LoginService.swift	Service	Appels API d'authentification
RefreshService.swift	Service	Gestion du refresh de session / token
SessionViewModel.swift	ViewModel	État global de session utilisateur
SessionManager.swift	Manager / Service	Orchestration session (login / logout / restore)
KeychainService.swift	Service	Accès sécurisé aux données sensibles
BackgroundView.swift	UI Component	Gestion du fond et du thème visuel
ForgotPasswordView.swift	Vue	Déclenchement du flux de réinitialisation

Navigation

Type	Détail
Entrée	Affichée dans le flow non authentifié (écran de connexion)
Sorties	Succès login -> interface authentifiée Mot de passe oublié -> ForgotPasswordView (sheet)
Conditions	Succès dépend d'une authentification valide
Dépendances	SessionViewModel / AppState

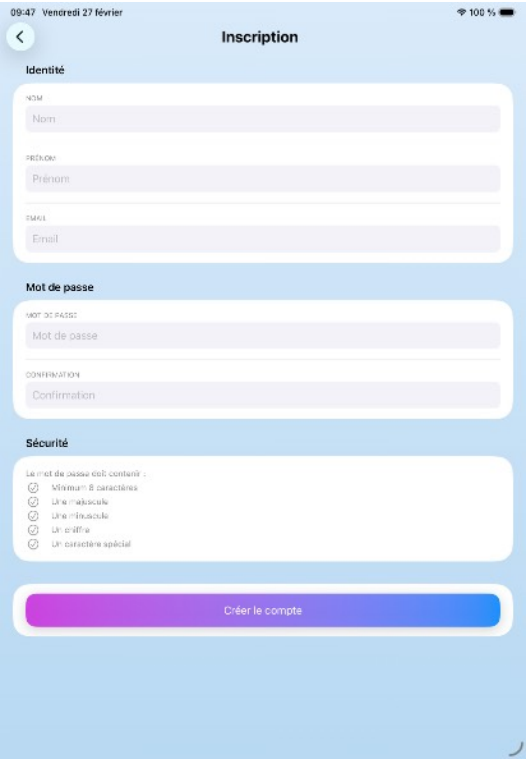
API

Domaine	Méthode	Endpoint	Description
Authentification	POST	/api/login_check	Authentification utilisateur (email / mot de passe)
Session	POST	/api/token/refresh	Rafraîchissement du token (session / biométrie)

Règles de gestion

ID	Règle
LOV01	La vue est visible uniquement lorsque l'utilisateur est **non authentifié**.
LOV02	L'email est obligatoire pour la connexion, un message d'erreur est affiché s'il est vide.
LOV03	Le password est obligatoire pour la connexion, un message d'erreur est affiché s'il est vide.
LOV04	Si le couple email/password est incorrect, le message d'erreur du backend est affiché.
LOV05	Une connexion réussi navigue vers l'écran MainView et BacklogView.
LOV06	Mot de passe oublié navigue vers ForgotPasswordView

8.8.1.4 – Page d’inscription



View :

RegisterView

Rôle

Permet à l’utilisateur de créer un compte ou de modifier ses informations selon le mode d’ouverture (create / edit).

Description

Formulaire utilisateur permettant la saisie / édition des informations de profil.
Affiche les erreurs de validation ou backend.
Les champs et le comportement varient selon le mode **création** ou **édition**.

Fichiers Swift associés

Élément	Type	Description
RegisterViewModel.swift	ViewModel	Gestion des états UI et validation du formulaire
RegisterService.swift	Service	Appels API de création / mise à jour utilisateur
SessionViewModel.swift	ViewModel	Mise à jour de l’état utilisateur après succès
BackgroundView.swift	UI Component	Gestion du fond et du thème visuel

Navigation

Type	Détail
Entrées	ConnexionView -> mode création ProfileView -> mode édition
Sorties	Succès création -> retour vers connexion / login automatique Succès modification -> retour écran précédent
Conditions	Comportement dépend du mode (create / edit)
Paramètres	mode (création / édition) user (en mode édition)

API

Domaine	Méthode	Endpoint	Description
Utilisateur	POST	/api/users/register	Création d'un nouveau compte utilisateur
Utilisateur	PATCH	/api/users/{id}	Mise à jour du profil utilisateur

Règles de gestion

ID	Règle
REV01	Le nom est obligatoire, un message d'erreur est affiché à la création s'il n'est pas saisi.
REV02	Le prénom est obligatoire, un message d'erreur est affiché à la création s'il n'est pas saisi.
REV03	L'email est obligatoire, un message d'erreur est affiché à la création s'il n'est pas saisi.
REV04	L'email doit avoir un format valide et doit être unique.
REV05	Le password et la confirmation sont obligatoires.
REV06	Le password et la confirmation doivent être identique.
REV07	Le password doit respecter les règles affichés à l'écran.
REV08	En cas d'erreur, le message du backend est affiché, ou à défaut erreur réseau s'il est injoignable.

8.8.2 – Backlog

8.8.2.1 – Vue principale

View :

MainView

Rôle

Vue principale post-authentification. Elle structure l'application en onglets et adapte les écrans disponibles selon le profil de l'utilisateur.

Description

Affiche l'interface principale via un TabView une fois l'utilisateur chargé.
Onglet **Backlog** toujours visible ; onglets **Admin** conditionnels (Manager).
Accès toolbar : **Profil** et **Créer projet**.
Affiche un loader pendant l'initialisation.

Fichiers Swift associés

Élément	Type	Description
SessionViewModel.swift	ViewModel	Gestion de l'état utilisateur et des droits
BacklogView.swift	Vue	Onglet Backlog (projets / user stories)
UsersListView.swift	Vue	Onglet Utilisateurs (visible pour Manager)
ProjectsView.swift	Vue	Onglet Projets (visible pour Manager)
ProfileView.swift	Vue	Écran Profil utilisateur
ProjectCreationView.swift	Vue	Création d'un nouveau projet
ProjectFormViewModel.swift	ViewModel	Logique et validation du formulaire projet
BackgroundView.swift	UI Component	Gestion du fond et du thème visuel

Navigation

Type	Détail
Entrée	Affichée après authentification (racine connectée)
Onglets	Backlog -> BacklogView Utilisateurs (Manager) -> UsersListView Projets (Manager) -> ProjectsView
Overlays	Profil -> ProfileView (sheet / fullScreenCover) Créer projet -> ProjectCreationView (fullScreenCover)
Conditions	Onglets Utilisateurs / Projets visibles pour profil Manager
Événements	Après création projet -> notification projectListShouldRefresh

API

Aucune.

Règles de gestion

ID	Règle
MAV01	Un profil utilisateur ne peut voir que Backlog, les 2 autres tabs sont visibles par un profil Manager.
MAV02	Le tab Backlog est affiché par défaut et navigue vers BacklogView.
MAV03	Le tab Utilisateurs navigue vers UsersListView.
MAV04	Le tab Projets navigue vers ProjectsView.

8.8.2.2 – Vue du Backlog

View :

BacklogView

Rôle

Affiche les projets actifs et permet de filtrer l'affichage par tags (avec impact sur projets + user stories affichées).

Description

Affiche le backlog des projets avec filtrage par tags.
Inclut l'indication du filtre actif, une barre de tags et un état **Filtrage…**.
Gère les états : chargement, erreur et résultat vide.

Fichiers Swift associés

Élément	Type	Description
ProjectListViewModel.swift	ViewModel	Logique backlog et filtrage des projets
SessionViewModel.swift	ViewModel	Gestion session utilisateur et droits
TagsService.swift	Service	Appels API liés aux tags et impacts
BacklogProjetsView.swift	Vue / UI Component	Affichage d'un projet dans le backlog
TagFilterChip.swift	UI Component	Filtrage des projets par tag
TagBadgeMiniView.swift	UI Component	Affichage du badge tag actif
BackgroundView.swift	UI Component	Fond visuel et thème écran
TagResponse	Modèles	Données tags
StoryStatus	Modèles	Données statuts
TagImpactResponse	Modèles	Données impact

Navigation

Type	Détail
Entrée	MainView -> onglet Backlog
Sorties	Aucune navigation directe (portée par les vues projet / story)
Conditions	-
Événements	.projectListShouldRefresh -> recharge projets actifs (hors filtrage) .userStoryDidChange -> relance filtrage actif ou recharge projets .tagsDidChange -> recharge liste des tags

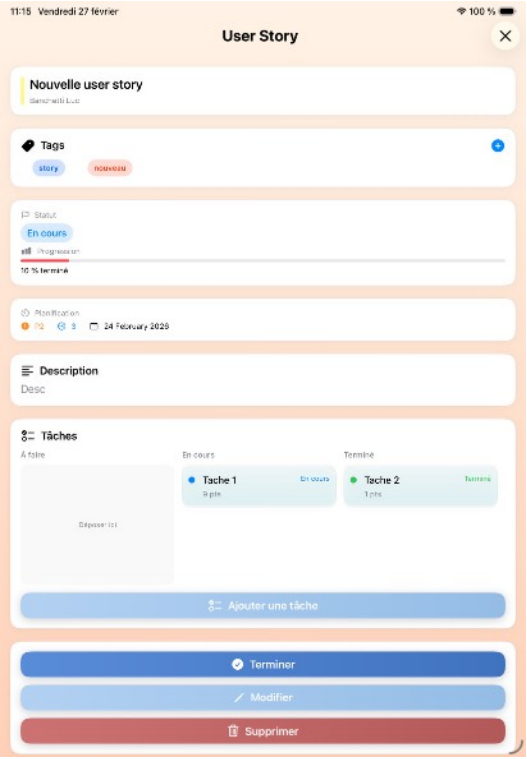
API

Domaine	Méthode	Endpoint	Description
Tags	GET	/api/tag/liste	Récupération des tags disponibles
Tags	GET	/api/tag/impact/{id}	Récupération de l'impact du tag (projets + stories)
Projets	GET	/api/projects/active*	Récupération des projets actifs

Règles de gestion

ID	Règle
BAV01	Le backlog affiche la liste des projets E**n cours** où l'utilisateur est **owner** du projet ou seulement **membre**, les autres projets ne sont pas visibles.
BAV02	Le filtre par tag active le mode filtre et remplace la liste par les résultats d'impact (user stories portant le tag où l'utilisateur est **membre** ou **owner** + le projet lié).
BAV03	L'utilisateur peut effectuer des drags and drop de User Story vers un autre état au sein du projet.
BAV04	L'utilisateur peut créer une nouvelle User Story sur un projet en cliquant sur **ajouter une User Story**, la navigation est effectué vers BacklogUSView.
BAV05	L'utilisateur peut consulter le détail du User Story en cliquant dessus, la navigation est alors fait vers UserStoryDetailView.
BAV06	Chaque projet contient la liste des user stories du projet, toutes si l'utilisateur est owner ou uniquement celle de l'utilisateur s'il n'est pas owner.
BAV07	La liste des tags contient une scrollview horizontale avec un mini badge de chaque tag de l'application ayant pour fond la couleur sélectionnée à la création du tag.

8.8.2.3 – Détail d’une User Story



View :

UserStoryDetailView

Rôle

Détail d’une User Story.

Description

Contenu d’une User Story, permet la modification, suppression, ajout de tag et tâches.

Fichiers Swift associés

Élément	Type	Description
UserStoriesService.swift	Service	Détail, modification, suppression
TagsService.swift	Service	Gestion des tags (attach / detach / impact)
TagBadgeView.swift	UI Component	Badge visuel des tags
ProgressTaskView.swift	UI Component	Affichage de la progression des tâches
DraggableTask.swift	UI Component	Gestion du drag & drop des User Stories
BackgroundView.swift	UI Component	Gestion du fond et du thème visuel

Navigation

Type	Détail
Entrée	Lien depuis BacklogView
Sortie	Croix ou suppression
Conditions	Mode dépend du rôle utilisateur
Règles	Owner -> mode édition Non-owner -> mode lecture seule

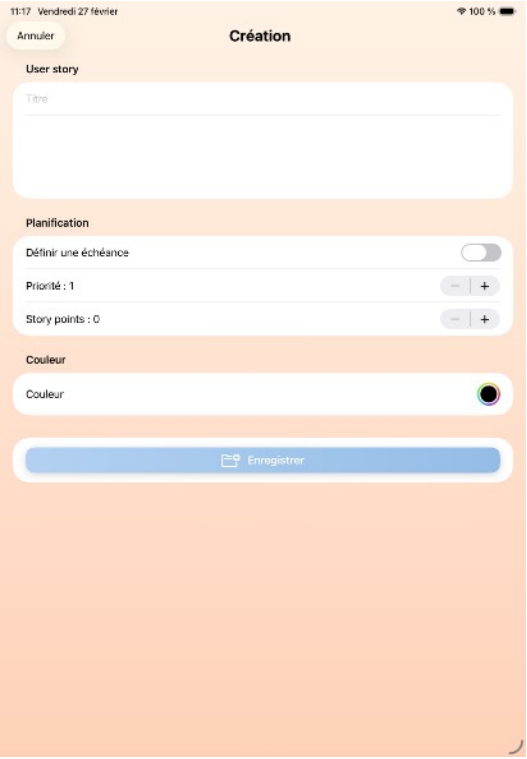
API

Domaine	Méthode	Endpoint	Description
Tags	DELETE	/api/tag/deltag/{tagId}/userstory/{storyId}	Détache un tag d'une User Story

Règles de gestion

ID	Règle
BUV01	La section tag affiche la liste des tags associées à la User Story.
BUV02	La section statut comprend, l'état actuel de la User Story. Une barre de progression dépendante des tâches à l'état terminé (pondéré par le poids des story points de la tâche).
BUV03	La section description comprend si elles ont été définis, les valeurs de priorités, de story points et de date de fin attendue.
BUV04	La section tâches affiche la liste des tâches de la User Story, l'utilisateur peut effectuer des drag and drop de la tâche vers un autre état.
BUV05	L'utilisateur owner peut créer une nouvelle tâche, la navigation est effectué vers TaskFormView.
BUV06	L'utilisateur owner peut modifier l'état aussi par un bouton vers son prochain état éligible.
BUV07	L'utilisateur owner peut modifier la User Story, la navigation est faite vers UserStoryFormView.
BUV08	L'utilisateur owner peut supprimer la User Story

8.8.2.4 – Création d'une User Story



View :

UserStoryFormView

Rôle

Permet la création ou la modification d'une User Story via un formulaire unique (mode create / edit).

Description

Formulaire permettant la création ou l'édition d'une User Story.
Gestion des erreurs de validation / API.
Action **Enregistrer** pour la persistance.

Fichiers Swift associés

Élément	Type	Description
UserStoryFormViewModel.swift	ViewModel	Gestion des états UI et validation du formulaire User Story
StoriesService.swift	Service	Appels API de création / mise à jour / statut
StoryResponse	Modèle	Représentation des données User Story
StoryStatus	Modèle	Statuts et workflow de la User Story
BackgroundView.swift	UI Component	Gestion du fond et du thème visuel

Navigation

Type	Détail
Entrées	Mode création Mode édition
Sorties	Succès enregistrement -> fermeture + callback Fermeture utilisateur -> dismiss()
Présentation	sheet / fullScreenCover
Conditions	Comportement dépend du mode

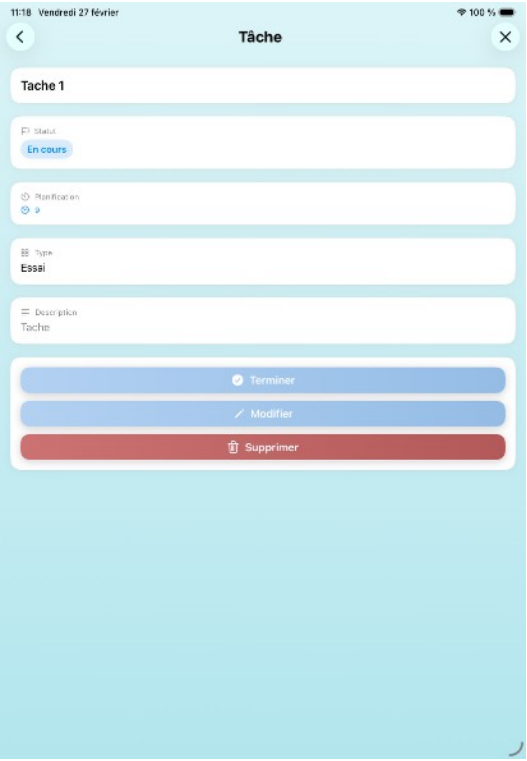
API

Domaine	Méthode	Endpoint	Description
User Stories	POST	/api/projects/{projectId}/userstories/create	Création d'une User Story
User Stories	PATCH	/api/userstories/{userStoryId}	Mise à jour d'une User Story

Règles de gestion

ID	Règle
USV01	Le champ **Titre** est obligatoire.
USV02	Le champ **Description** est obligatoire.
USV03	La **date d'échéance** est facultative, elle est affiché si l'utilisateur a sélectionné le toggle button.
USV04	En mode **édition**, les champs sont **pré-remplis**.

8.8.2.5 – Détail d’une tâche



View :

TaskDetailView

Rôle

Afficher le détail d’une tâche et permettre sa gestion (consultation, mise à jour, suppression).

Description

Écran ouvert à partir du Kanban des tâches d’une User Story (tap sur une carte).
La vue charge les informations de la tâche via son taskId puis propose les actions usuelles (au minimum : voir les infos, changer le statut, éditer et/ou supprimer selon droits).

Fichiers Swift utilisés

Élément	Type	Description
TaskService.swift	Service	Appels API réseau liés aux tâches
APITask.swift	Modèles	Définition des structures (TaskResponse, TaskStatus, etc.)
TaskFormView.swift	Vue	Écran création / édition tâche
TaskFormViewModel.swift	ViewModel	Logique formulaire et validation
TasksSectionView.swift	UI Component	Section Kanban contenant les colonnes
TaskColumnView.swift	UI Component	Affichage d’une colonne Kanban
TaskCardView.swift	UI Component	Carte tâche, point d’entrée navigation détail

Navigation

Type	Détail
Entrée	TasksSectionView -> TaskDetailView
Déclencheur	Sélection d'une tâche
Sorties	Retour écran précédent
Événements	.taskDidChange -> rafraîchissement du Kanban

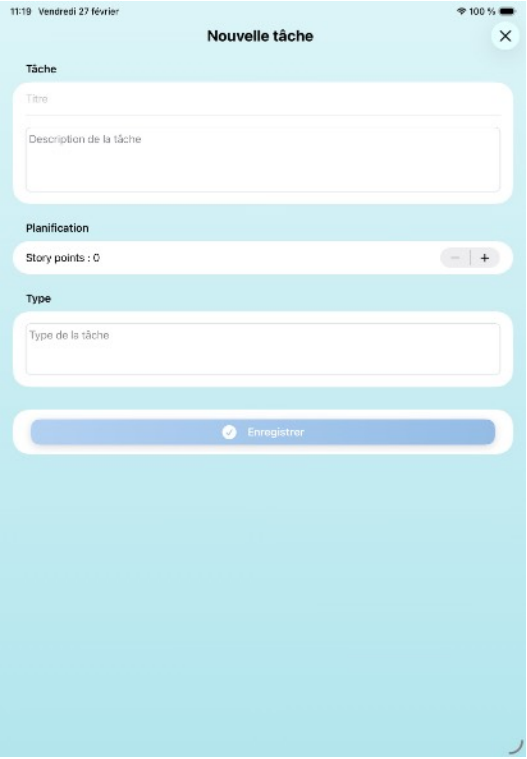
API

Domaine	Méthode	Endpoint	Description
Tâches	GET	/api/tasks/detail/{taskId}	Récupération du détail d'une tâche
Tâches	PATCH	/api/tasks/status/{taskId}	Changement de statut
Tâches	PATCH	/api/tasks/update/{taskId}	Mise à jour d'une tâche
Tâches	DELETE	/api/tasks/delete/{taskId}	Suppression d'une tâche

Règles de gestion

ID	Règle
TDV01	L'utilisateur peut modifier l'état de la tâche vers son prochain état elligible.
TDV02	L'utilisateur peut modifier la tâche, la navigation est effectuée vers TaskFormView.
TDV03	L'utilisateur peut supprimer la tâche.

8.8.2.6 – Création d’une tâche



View :

TaskFormView

Rôle

Permet la création ou la modification d’une tâche via un formulaire unique (mode create / edit).

Description

Formulaire permettant la création ou l’édition d’une tâche.
Gestion des erreurs de validation / API.
Action **Enregistrer** pour la persistance.

Fichiers Swift associés

Élément	Type	Description
TaskFormViewModel.swift	ViewModel	Gestion des états UI et validation du formulaire tâche
TaskService.swift	Service	Appels API de création / mise à jour tâche
TaskResponse	Modèle	Représentation des données tâche
TaskStatus	Modèle	Statuts et workflow de la tâche
TaskMode	Modèle	Mode du formulaire (create / edit)
BackgroundView.swift	UI Component	Gestion du fond et du thème visuel

Navigation

Type	Détail
Entrées	Mode création Mode édition
Sorties	Succès enregistrement -> fermeture Fermeture utilisateur -> dismiss()
Conditions	Comportement dépend du mode
Événements	Succès -> notification .taskDidChange

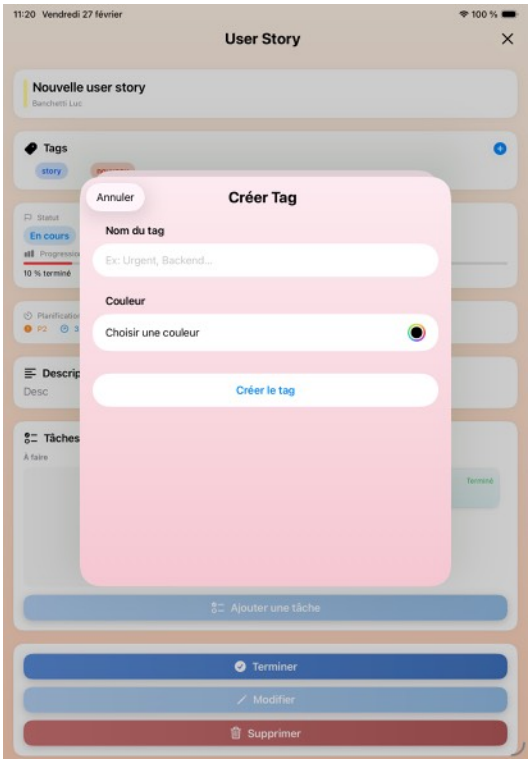
API

Domaine	Méthode	Endpoint	Description
Tâches	POST	/api/userstories/{userStoryId}/tasks/create	Création d'une tâche
Tâches	PATCH	/api/tasks/update/{taskId}	Mise à jour d'une tâche

Règles de gestion

ID	Règle
TFV01	Le champ **Titre** est obligatoire.
TFV02	Le champ **Description** est obligatoire.
TFV03	Le champ **Type** est obligatoire.
TFV04	Le champ **Story points** est facultatif.
TFV05	En mode **édition**, les champs sont **pré-remplis**.
TFV06	Après un enregistrement réussi, la vue est **fermée automatiquement**.
TFV07	En cas d'erreur, un **message d'erreur** est affiché.

8.8.2.7 – Création d'un tag



View :

TagFormView

Rôle

Permet la création ou la modification d'un tag (nom + couleur) via un formulaire unique.

Description

Formulaire de création ou modification de tag (nom + couleur).
Gestion des erreurs et état de chargement.
Succès: fermeture de la vue.

Fichiers Swift associés

Élément	Type	Description
TagFormViewModel.swift	ViewModel	Gestion des états UI et validation du formulaire tag
TagsService.swift	Service	Appels API de création / mise à jour tag
TagMode	Modèle	Mode du formulaire (create / edit)
TagResponse	Modèle	Représentation des données tag
Color(hex:)	Utilitaire	Conversion couleur hex → SwiftUI Color
toHex()	Utilitaire	Conversion SwiftUI Color → hex
.adaptiveTextColor()	UI Modifier	Adaptation couleur texte au thème
BackgroundView.swift	UI Component	Gestion du fond et du thème visuel

Navigation

Type	Détail
Entrées	Initialisation via TagFormView (mode création / édition)
Sorties	Annuler -> dismiss() Succès -> fermeture
Présentation	sheet / fullScreenCover
Conditions	Comportement dépend du mode

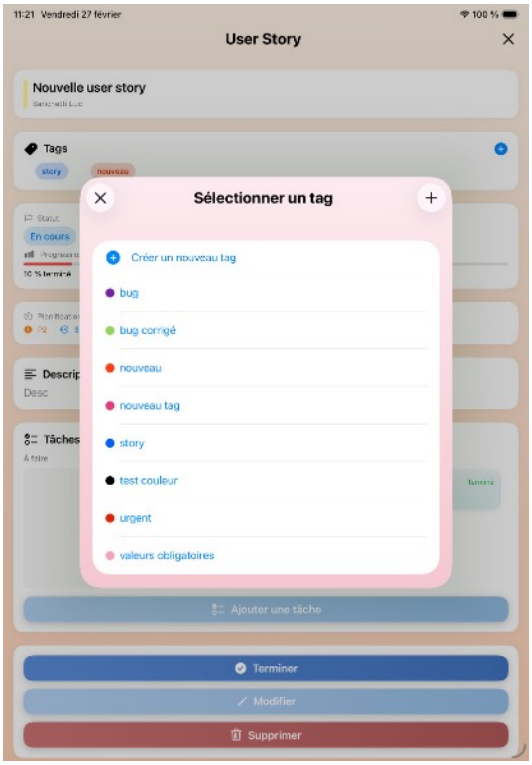
API

Domaine	Méthode	Endpoint	Description
Tags	POST	/api/tag/create	Création d'un tag
Tags	PATCH	/api/tag/update/{tagId}	Mise à jour d'un tag

Règles de gestion

ID	Règle
TAV01	Le titre de navigation affiche **"Créer Tag"** ou **"Modifier Tag"**.
TAV02	Le bouton affiche **"Créer le tag"** ou **"Mettre à jour"**.
TAV03	Si enregistrement avec succés, fermeture automatique de la vue.
TAV04	En cas d'erreur, un message est affiché dans une section dédiée.

8.8.2. – Sélection d’un tag



View :

TagPickerView

Rôle

Permet à l’utilisateur de sélectionner un tag existant et, si besoin, de créer rapidement un nouveau tag.

Description

Permet la sélection d’un tag existant ou la création rapide d’un nouveau tag.
Gestion des états : chargement, erreur et liste vide.
Sélection: fermeture de la vue.

Fichiers Swift associés

Élément	Type	Description
TagPickerViewModel.swift	ViewModel	Gestion du chargement des tags et des états UI
TagsService.swift	Service	Appels API tags (via le ViewModel)
TagFormView.swift	Vue	Création rapide d’un nouveau tag
TagRowView.swift	UI Component	Affichage d’une ligne tag
BackgroundView.swift	UI Component	Fond visuel et thème écran
TagResponse	Modèle	Représentation des données tag

Navigation

Type	Détail
Entrée	Présentée en sheet depuis une vue consommatrice
Sorties	Sélection tag -> fermeture Fermeture utilisateur -> dismiss() Création -> TagFormView (sheet)
Conditions	-
Événements	Après création -> rechargement + notification .tagsDidChange

API

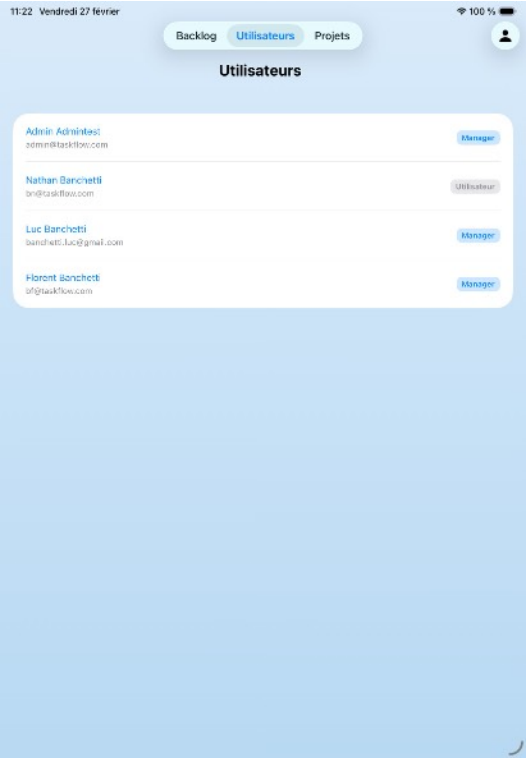
Domaine	Méthode	Endpoint	Description
Tags	GET	/api/tag/liste	Récupération des tags disponibles
Tags	POST	/api/tag/create	Création d'un tag

Règles de gestion

ID	Règle
TLV01	L'utilisateur peut créer un tag, la navigation est faite vers TagFormView.
TLV02	L'utilisateur peut sélectionner un tag à ajouter à la User Story en cliquant dessus.

8.8.3 – Utilisateurs

8.8.3.1 – Liste des utilisateurs



View :

UsersListView

Rôle

Afficher la liste des utilisateurs et permettre l'accès au détail d'un utilisateur.

Description

Affiche la liste des utilisateurs avec gestion des états : chargement, erreur et vide.
La sélection permet la navigation vers le détail utilisateur.

Fichiers Swift associés

Élément	Type	Description
UsersListViewModel.swift	ViewModel	Gestion des états UI et du chargement de la liste utilisateurs
UsersListService.swift	Service	Appels API de récupération des utilisateurs
ProfileRowView.swift	UI Component	Affichage d'une ligne utilisateur
UserDetailView.swift	Vue	Écran de détail d'un utilisateur
ProfileResponse	Modèle	Représentation complète des données utilisateur
ProfileLiteResponse	Modèle	Représentation simplifiée utilisateur
BackgroundView.swift	UI Component	Gestion du fond et du thème visuel

Navigation

Type	Détail
Entrée	MainView -> onglet Utilisateurs
Sortie	Sélection utilisateur -> UserDetailView
Conditions	Onglet visible pour profil Manager

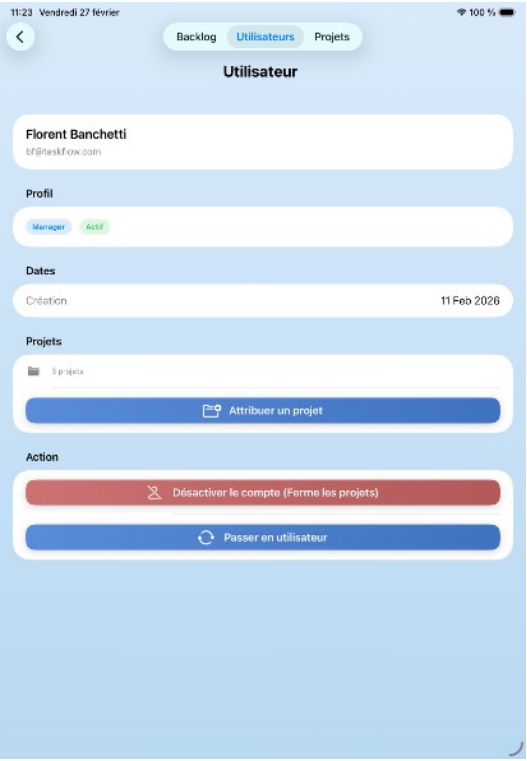
API

Domaine	Méthode	Endpoint	Description
Utilisateurs	GET	/api/users/liste	Récupération de la liste des utilisateurs

Règles de gestion

ID	Règle
ULV01	Affiche la liste des utilisateurs actifs.
ULV02	L'utilisateur peut consulter le détail d'un profil en cliquant dessus, la navigation est effectué vers UserDetailView.

8.8.3.2 – Détail d'un utilisateur



View :

UserDetailView

Rôle

Afficher le détail d'un utilisateur (identité, rôle, statut, projets) et fournir des actions d'administration (réservées aux managers).

Description

Affiche le détail d'un utilisateur : identité, rôle, statut et informations du compte.
Permet l'accès à la gestion des projets associés.
Propose des actions d'administration (**activation / désactivation**, **changement de rôle**) visibles uniquement pour les managers.

Fichiers Swift associés

Élément	Type	Description
UserAdminViewModel.swift	ViewModel	Gestion des actions administrateur et des états UI
UserAdminService.swift	Service	Appels API d'administration utilisateur
UserProjectsView.swift	Vue	Écran d'attribution / consultation des projets (sheet)
UserProjectsViewModel.swift	ViewModel	Logique de chargement / attribution projets utilisateur
OwnerProjectsViewModel.swift	ViewModel	Gestion des projets liés au user courant (owner)
ProjectService.swift	Service	Appels API projets / membres / attribution
BackgroundView	UI Component	Fond visuel et thème écran
BoutonImageView	UI Component	Boutons d'actions admin
ProfileResponse	Modèle	Représentation des données utilisateur
AdminUserStatus	Modèle	Statuts administrables (actif / inactif…)
AdminUserProfil	Modèle	Rôles administrables (manager / utilisateur…)

Navigation

Type	Détail
Entrée	UsersListView ->UserDetailView
Sorties	Attribuer projet -> UserProjectsView (sheet)
Confirmations	Activation / désactivation Changement de rôle
Conditions	Actions admin visibles pour profil Manager

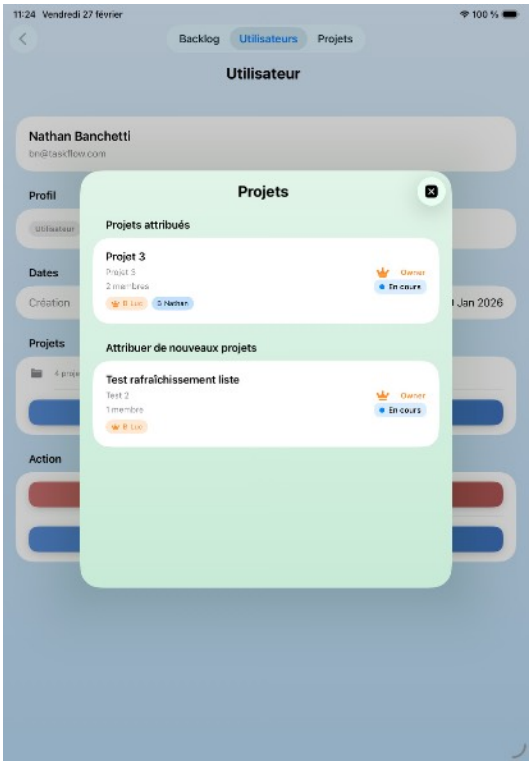
API

Domaine	Méthode	Endpoint	Description
Administration	PATCH	/api/admin/update/{userId}	Mise à jour du statut / rôle utilisateur
Projets	GET	/api/projects/user/{userId}*	Récupération des projets d'un utilisateur
Projets	POST	/api/projects/{projectId}/members/add/{userId}*	Ajout d'un membre au projet

Règles de gestion

ID	Règle
UDV01	L'utilisateur peut attribuer des projets dont il est owner, la navigation est faite vers UserProjetsView
UDV02	L'utilisateur peut désactiver ou réactiver le compte.
UDV03	L'utilisateur peut modifier les droits du compte.
UDV04	Le changement de rôle (**Manager : Utilisateur**) nécessite une **confirmation**.
UDV05	Les boutons n'actions ne sont pas visibles si l'utilisateur consulte son profil.

8.8.3.3 – Projets de l’utilisateur



View :

UserProjectsView

Rôle

Permet de consulter les projets associés à un utilisateur et d’attribuer un projet à cet utilisateur.

Description

Affiche les projets associés à un utilisateur.
Permet l’attribution d’un projet.
Gestion des états : chargement, erreur et vide.

Fichiers Swift associés

Élément	Type	Description
UserProjectsViewModel.swift	ViewModel	Gestion du chargement et de l’attribution des projets utilisateur
ProjectService.swift	Service	Appels API liés aux projets / membres
ProjectResponse	Modèle	Représentation des données projet
BackgroundView.swift	UI Component	Gestion du fond et du thème visuel

Navigation

Type	Détail
Entrée	UserDetailView
Présentation	sheet
Sorties	Fermeture -> dismiss()
Conditions	Retour automatique possible après action

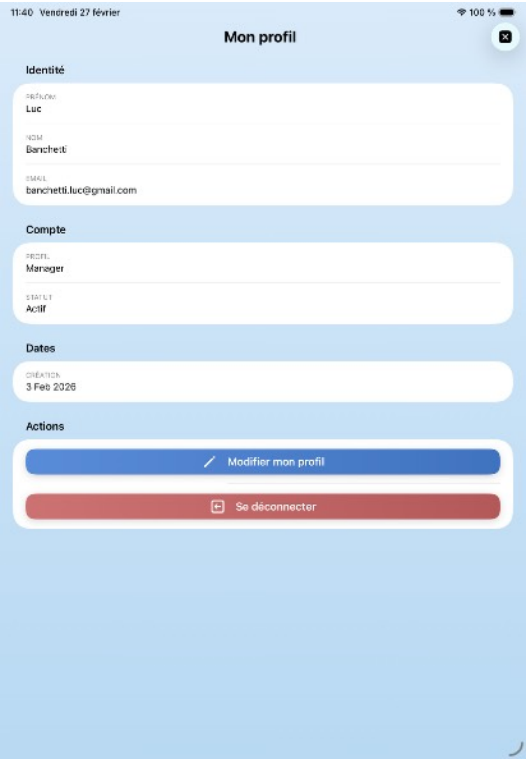
API

Domaine	Méthode	Endpoint	Description
Projets	GET	/api/projects/user/{userId}	Récupération des projets associés à un utilisateur
Projets	POST	/api/projects/projet/{projectId}/member/{userId}	Ajout d'un utilisateur comme membre du projet

Règles de gestion

ID	Règle
UPV01	La première section affiche les projets déjà attribués au compte utilisateur.
UPV02	La seconde section affiche les projets non attribués au compte utilisateur et dont la personne connectée est owner.
UPV03	Via un swipe dans la seconde liste, le manager peut attribuer des projets.

8.8.3.4 – Profil utilisateur



View :

ProfileView

Rôle

Afficher le profil de l'utilisateur connecté et proposer les actions principales : modifier le profil et se déconnecter.

Description

Affiche les informations du profil utilisateur et les actions associées (**modifier**, **se déconnecter**).
Gestion des états : chargement et données indisponibles.

Fichiers Swift associés

Élément	Type	Description
SessionViewModel.swift	ViewModel	Gestion de l'état utilisateur et des actions de session
AppState.swift	State	Gestion du flow global et de la navigation
RegisterView.swift	Vue	Écran d'édition du profil utilisateur
BackgroundView	UI Component	Fond visuel et thème écran
BoutonImageView	UI Component	Boutons d'action (modifier / logout)

Navigation

Type	Détail
Entrée	MainView
Présentation	sheet / fullScreenCover
Sorties	Fermeture -> dismiss() Modifier profil -> RegisterView (mode édition) Se déconnecter -> retour flow login
Conditions	-

API

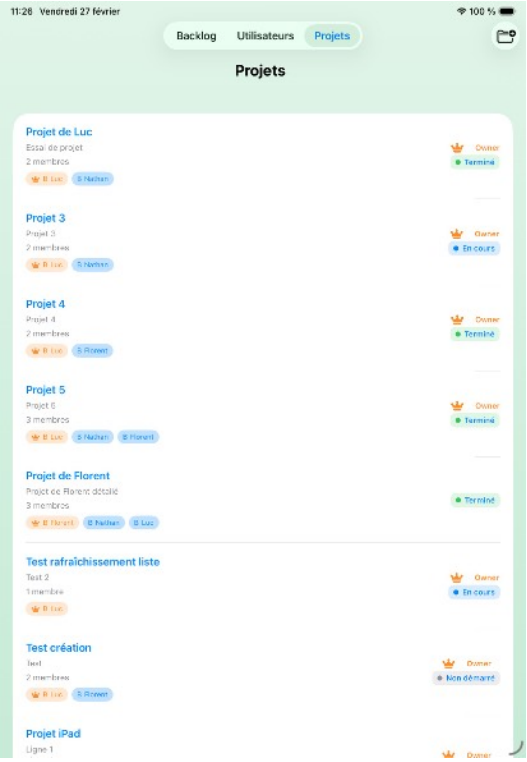
Aucune.

Règles de gestion

ID	Règle
PLV01	Une personne peut consulter son profil depuis le backlog ou la liste des utilisateurs
PLV02	Il peut modifier le contenu ou le mot de passe, la navigation est effectuée vers RegisterView.
PLV03	Il peut aussi se déconnecter, le token est alors effacé et il devra alors entrer ses identifiants la prochaine fois.

8.8.4 – Projets

8.8.4.1 – Liste des projets



View :

ProjectsView

Rôle

Afficher la liste des projets accessibles à l'utilisateur (membre ou owner) et permettre la navigation vers le détaild'un projet.

Description

Affiche la liste des projets accessibles avec gestion des états : chargement, erreur et vide.
Sélection -> navigation vers le détail du projet.

Fichiers Swift associés

Élément	Type	Description
ProjectListViewModel.swift	ViewModel	Gestion du chargement et des états UI de la liste projets
ProjectService.swift	Service	Appels API liés aux projets
ProjectsRowView.swift	UI Component	Affichage d'une ligne projet
ProjectDetailView.swift	Vue	Écran de détail d'un projet
ProjectDetailViewModel.swift	ViewModel	Logique du détail projet et actions associées
ProjectResponse	Modèle	Représentation des données projet
ProjectStatus	Modèle	Statuts et workflow du projet
BackgroundView.swift	UI Component	Gestion du fond et du thème visuel

Navigation

Type	Détail
Entrée	MainView -> onglet Projets
Sortie	Sélection projet -> ProjectDetailView
Conditions	Onglet visible pour profil Manager
Rafraîchissement	Chargement écran -> fetch projets Notification .projectListShouldRefresh -> refresh

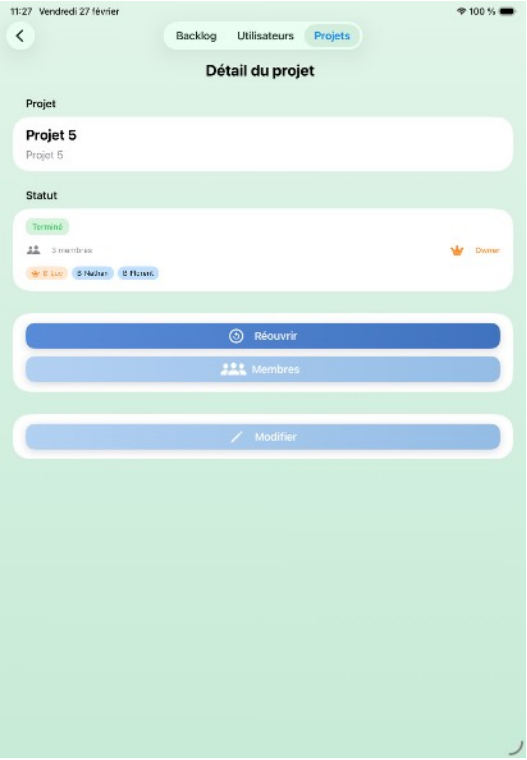
API

Domaine	Méthode	Endpoint	Description
Projets	GET	/api/projects/liste	Récupération des projets accessibles

Règles de gestion

ID	Règle
PLV01	La liste affiche les projets dont l'utilisateur est owner ou membre.
PLV02	L'utilisateur peut créer un projet, la navigation est effectuée vers ProjectCreationView.
PLV03	L'utilisateur peut consulter un projet en cliquant dessus, la navigation est effectuée vers ProjectDetailView.

8.8.4.2 – Détail d’un projet



View :

ProjectDetailView

Rôle

Afficher le détail d’un projet (infos, statut, membres) et permettre au propriétaire de le gérer : changer le statut, modifier, gérer les membres et supprimer (sous conditions).

Description

Affiche le détail d’un projet : informations, statut et membres.
Propose des actions conditionnelles pour le propriétaire (**changer statut**, **modifier**, **gérer les membres**, **supprimer**).
Inclut la gestion des états et des interactions (édition, sheets, confirmation, rechargement).

Fichiers Swift associés

Élément	Type	Description
ProjectDetailViewModel.swift	ViewModel	Gestion du détail projet et des actions (reload / statut / suppression)
ProjectService.swift	Service	Appels API liés au projet
ProjectEditView.swift	Vue	Écran d’édition du projet
ProjectUsersView.swift	Vue	Gestion des membres du projet
BackgroundView	UI Component	Fond visuel et thème écran
BoutonImageView	UI Component	Boutons d’action (statut / édition / suppression)
OwnerBadgeView	UI Component	Badge visuel du propriétaire
MemberBadgeView	UI Component	Badge visuel des membres
ProjectResponse	Modèle	Représentation des données projet
ProjectStatus	Modèle	Statuts du projet

Navigation

Type	Détail
Entrée	ProjectsView -> ProjectDetailView
Sorties	Modifier -> ProjectEditView (sheet) Membres -> ProjectUsersView (sheet) Supprimer -> fermeture après confirmation
Confirmations	Suppression projet
Événements	.projectDidChange -> rechargement du projet
Conditions	Actions limitées selon rôle / statut

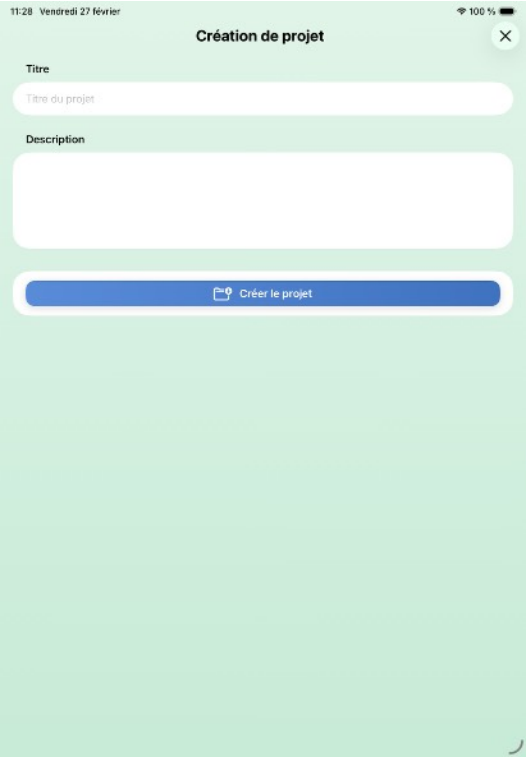
API

Domaine	Méthode	Endpoint	Description
Projets	GET	/api/projects/detail/{id}	Récupération du détail d'un projet
Projets	PATCH	/api/projects/status/{id}	Mise à jour du statut du projet
Projets	DELETE	/api/projects/delete/{id}	Suppression d'un projet

Règles de gestion

ID	Règle
PDV01	L'utilisateur owner peut modifier l'état du projet vers son prochain état éligible.
PDV02	L'utilisateur owner peut modifier la liste des personnes membres du projet, la navigation est effectuée vers ProjectUsersView
PDV03	L'utilisateur peut modifier le projet, la navigation est effectuée vers ProjectEditView.

8.8.4.3 – Création d’un projet



View :

ProjectCreationView

Rôle

Permet de créer un nouveau projet (titre + description) depuis un formulaire simple.

Description

Formulaire permettant la création d’un projet (titre + description).
Gestion des états : chargement, erreur et succès.
Succès: fermeture automatique.

Fichiers Swift associés

Élément	Type	Description
ProjectFormViewModel.swift	ViewModel	Gestion des états UI et validation du formulaire
ProjectService.swift	Service	Appels API de création / mise à jour projet
BackgroundView	UI Component	Fond visuel et thème écran
BoutonImageView	UI Component	Bouton principal d’action
ProjectResponse	Modèle	Représentation des données projet

Navigation

Type	Détail
Entrée	MainView -> ProjectCreationView
Présentation	fullScreenCover
Sorties	Fermeture -> dismiss() Succès création -> fermeture
Conditions	Succès dépend de la validation / API

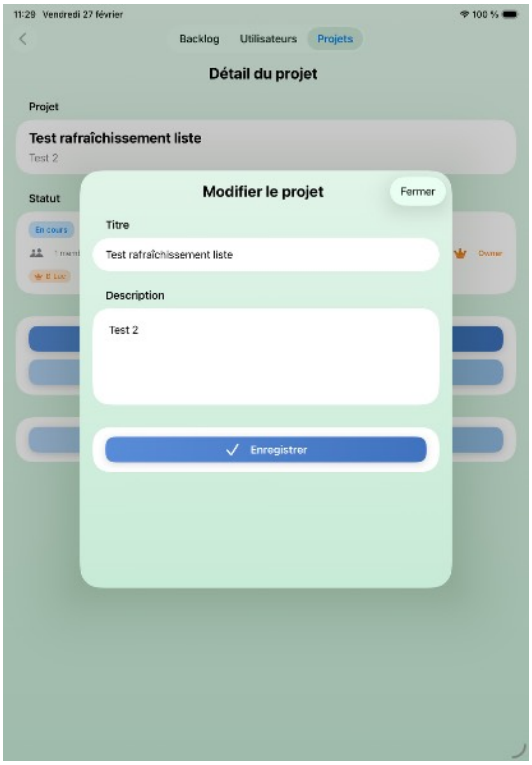
API

Domaine	Méthode	Endpoint	Description
Projets	POST	/api/projects/create	Création d'un projet

Règles de gestion

ID	Règle
PCV01	Le champ Titre est obligatoire.
PCV02	Le champ Description est obligatoire.
PCV03	En cas d'erreur, un message s'affiche.
PCV04	Si succès, fermeture automatique de la vue.

8.8.4.4 – Modification d’un projet



View :

ProjectEditView

Rôle

Permet de modifier un projet existant (titre + description).

Description

Formulaire permettant la modification d’un projet existant.
Gestion des états : erreur, chargement et succès.
Succès: fermeture de la vue.

Fichiers Swift associés

Élément	Type	Description
ProjectFormViewModel.swift	ViewModel	Gestion des états UI et validation du formulaire projet
ProjectService.swift	Service	Appels API de création / mise à jour projet
BackgroundView	UI Component	Fond visuel et thème écran
BoutonImageView	UI Component	Bouton principal d’action (création / sauvegarde)
.appNavigationTitle	UI Modifier	Gestion du titre de navigation
ProjectResponse	Modèle	Représentation des données projet
ProjectMode	Modèle	Mode du formulaire (create / edit)

Navigation

Type	Détail
Entrée	ProjectDetailView -> ProjectEditView
Présentation	sheet
Sorties	Annuler / Fermer -> dismiss() Succès sauvegarde -> fermeture
Conditions	-

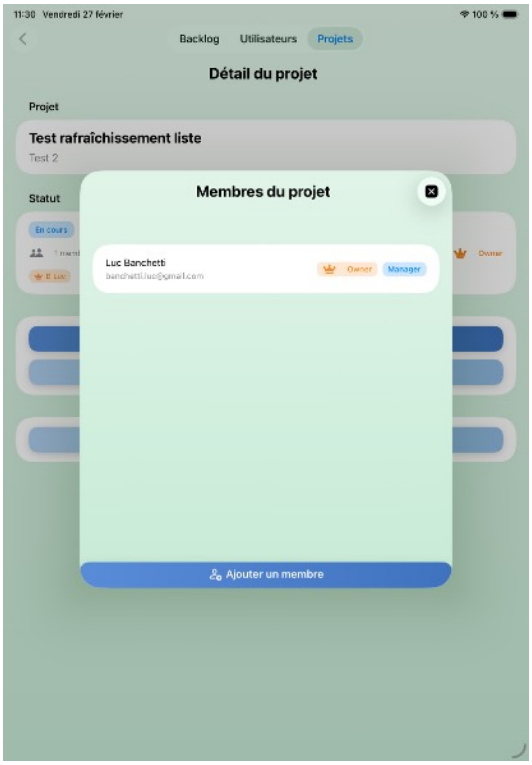
API

Domaine	Méthode	Endpoint	Description
Projets	PATCH	/api/projects/update/{id}	Mise à jour d'un projet

Règles de gestion

ID	Règle
PEV01	Les champs Titre et Description sont préremplis automatiquement.
PEV02	Le champ Titre est obligatoire.
PEV03	Le champ Description est obligatoire.
PEV04	En cas d'erreur, un message s'affiche.
PEV05	Si succès, fermeture automatique de la vue.

8.8.4.5 – Utilisateurs d'un projet



View :

ProjectUsersView

Rôle

Permet de gérer les membres d'un projet : consulter la liste, ajouter un utilisateur et supprimer un membre (hors owner).

Description

Affiche les membres d'un projet avec gestion des états : chargement, erreur et vide.
Permet l'ajout et la suppression de membres.
Indication visuelle du propriétaire.

Fichiers Swift associés

Élément	Type	Description
ProjectUsersViewModel.swift	ViewModel	Gestion des membres du projet et des états UI
ProjectService.swift	Service	Appels API liés aux membres du projet
ProjectUserSelectionView.swift	Vue	Sélection d'un utilisateur à ajouter
UsersListViewModel.swift	ViewModel	Chargement / filtrage de la liste utilisateurs
ProfileRowView	UI Component	Affichage d'un membre / utilisateur
BoutonImageView	UI Component	Boutons d'action stylisés
BackgroundView	UI Component	Fond visuel et thème écran
ProjectResponse	Modèle	Représentation des données projet
ProfileResponse	Modèle	Représentation des données utilisateur

Navigation

Type	Détail
Entrée	ProjectDetailView -> ProjectEditView
Présentation	sheet
Sorties	Annuler / Fermer -> dismiss() Succès sauvegarde -> fermeture
Conditions	-

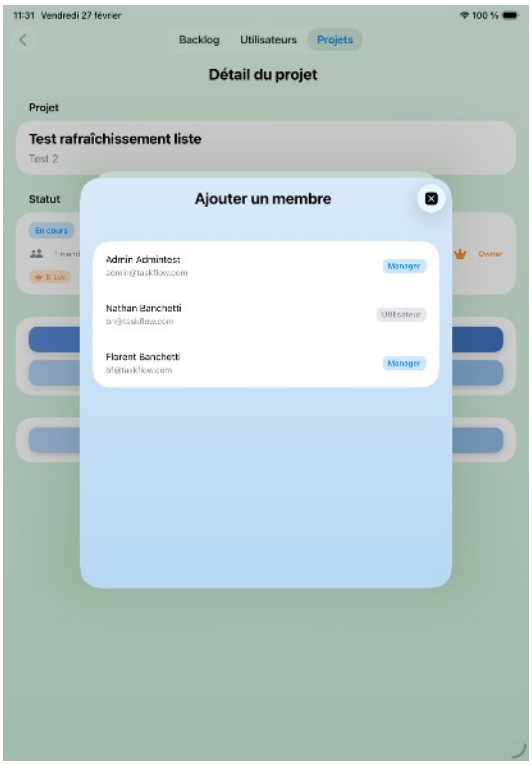
API

Domaine	Méthode	Endpoint	Description
Projets	GET	/api/projects/{projectId}/members	Récupération des membres du projet
Projets	POST	/api/projects/projet/{projectId}/member/{userId}	Ajout d'un membre au projet
Projets	DELETE	/api/projects/projet/{projectId}/member/{userId}	Suppression d'un membre du projet

Règles de gestion

ID	Règle
PMV01	Le chargement initial des **membres du projet** est déclenché automatiquement.
PMV02	Les membres sont affichés selon un **tri alphabétique**.
PMV03	Le Owner ne peut pas être supprimé.
PMV04	L'ajout d'un membre est autorisé via une action dédiée.
PMV05	Les utilisateurs **déjà membres** sont exclus de la sélection d'ajout.
PMV06	Après un ajout réussi -> la liste des membres est **rechargée**.
PMV07	Après suppression réussie -> la liste est **rechargée**.
PMV08	En cas d'échec API -> un **message d'erreur** est affiché.

8.8.4.6 - Sélection d’utilisateur sur un projet



View :

ProjectUserSelectionView

Rôle

Permet de sélectionner un utilisateur à ajouter comme membre d’un projet, en excluant ceux déjà membres.

Description

Permet la sélection d’utilisateurs à ajouter au projet. Exclut les membres existants et gère les états : chargement, erreur et vide.

Fichiers Swift associés

Élément	Type	Description
UsersListViewModel.swift	ViewModel	Gestion des états UI et du chargement des utilisateurs
UsersListService.swift	Service	Appel API de récupération des utilisateurs
ProfileRowView	UI Component	Affichage d’une ligne utilisateur
BackgroundView	UI Component	Fond visuel et thème écran
ProfileResponse	Modèle	Représentation des données utilisateur

Navigation

Type	Détail
Entrée	ProjectUsersView -> ProjectUserSelectionView
Présentation	sheet (dans NavigationStack)
Sorties	Sélection utilisateur -> fermeture Fermeture utilisateur -> dismiss()
Conditions	-

API

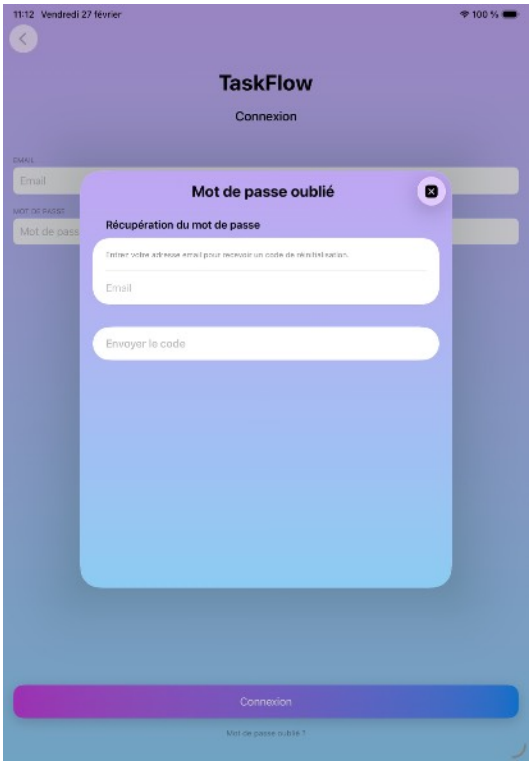
Domaine	Méthode	Endpoint	Description
Utilisateurs	GET	/api/users/liste	Récupération de la liste des utilisateurs

Règles de gestion

ID	Règle
PAV01	Liste les utilisateurs non membre d'un projet
PAV02	Les utilisateurs sont affichés selon un tri alphabétique par nom.
PAV03	Un tap sur un utilisateur déclenche l'ajout du membre.
PAV04	Un swipe "Ajouter" déclenche l'ajout du membre.

8.8.5 - Gestion des mots de passe

8.8.5.1 - Mot de passe oublié



View :

ForgotPasswordView

Rôle

Permet à l'utilisateur de demander un code de réinitialisation de mot de passe via son email, puis d'accéder à l'étape de validation du code.

Description

Permet la demande d'un code de réinitialisation via email.
Gestion des états : chargement et erreur.
Succès: navigation vers la validation du code.

Fichiers Swift associés

Élément	Type	Description
ForgotPasswordViewModel.swift	ViewModel	Gestion de la demande d'envoi du code et des états UI
ResetCodeView.swift	Vue	Écran de saisie et validation du code de réinitialisation
AuthService.swift	Service	Appels API de reset / vérification
BackgroundView	UI Component	Fond visuel et thème écran

Navigation

Type	Détail
Entrée	LoginView -> ForgotPasswordView
Présentation	sheet
Sorties	Fermeture -> dismiss() Succès -> ResetCodeView
Conditions	Succès dépend d'un envoi valide

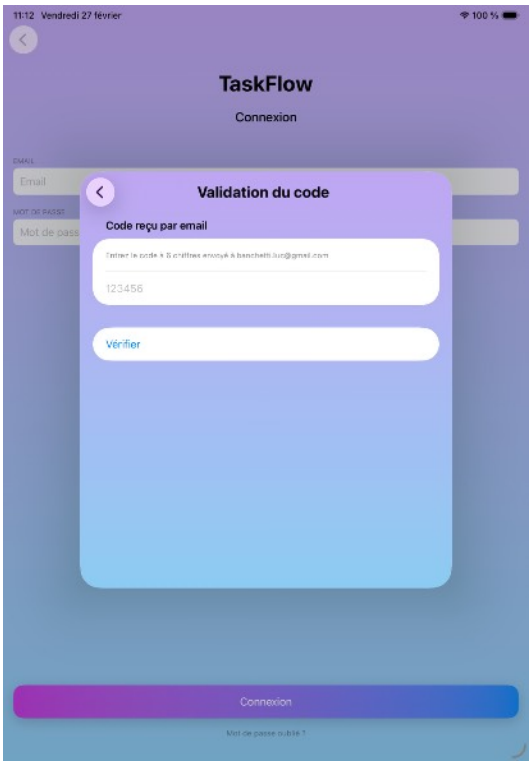
API

Domaine	Méthode	Endpoint	Description
Authentification	POST	/api/auth/reset-code	Demande d'un code de réinitialisation

Règles de gestion

ID	Règle
FPV01	Envoi un email si l'email est connu.
FPV02	En cas de succès -> navigation vers ResetCodeView.
FPV03	En cas d'erreur, affiche le message backend si disponible.

8.8.5.2 - Validation du code



View :

ResetCodeView

Rôle

Permet de valider le code de réinitialisation reçu par email avant d'accéder à la saisie d'un nouveau mot de passe.

Description

Permet la validation du code de réinitialisation.
Gestion des états : chargement et erreur.
Succès: navigation vers la définition du nouveau mot de passe.

Fichiers Swift associés

Élément	Type	Description
ResetCodeViewModel.swift	ViewModel	Gestion de la validation du code et des états UI
NewPasswordView.swift	Vue	Écran de saisie du nouveau mot de passe
NewPasswordViewModel.swift	ViewModel	Logique de reset et validation du formulaire
AuthService.swift	Service	Appels API de vérification / réinitialisation
BackgroundView	UI Component	Fond visuel et thème écran

Navigation

Type	Détail
Entrée	ForgotPasswordView -> ResetCodeView
Sorties	Succès vérification -> NewPasswordView
Retour	Géré par la NavigationStack parente
Conditions	Succès dépend d'un code valide

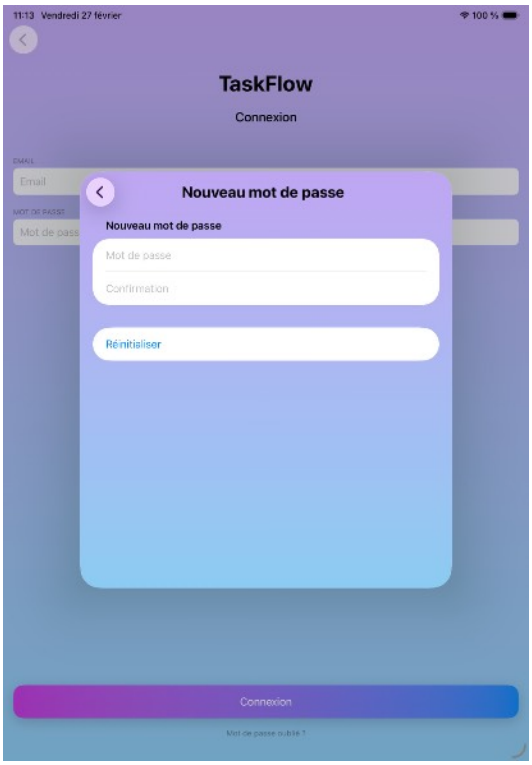
API

Domaine	Méthode	Endpoint	Description
Authentification	POST	/api/auth/reset/verify	Vérification du code et récupération du token
Authentification	POST	/api/auth/reset-code	Redemande d'un code de réinitialisation

Règles de gestion

ID	Règle
RCV01	Si le code entrée par l'utilisateur correspond à celui adressé par mail et qu'il est encore valide, la navigation est effectuée vers NewpassWordView

8.8.5.3 - Mise à jour du mot de passe



View :

NewPasswordView

Rôle

Permet de définir un nouveau mot de passe à partir d'un token obtenu après validation du code de réinitialisation.

Description

Permet la définition d'un nouveau mot de passe après validation du reset.
Gestion des états : chargement et erreur.
Succès: fermeture automatique de la vue.

Fichiers Swift associés

Élément	Type	Description
NewPasswordViewModel.swift	ViewModel	Gestion des états UI et logique de reset du mot de passe
AuthService.swift	Service	Appels API de validation / réinitialisation
BackgroundView	UI Component	Fond visuel et thème écran

Navigation

Type	Détail
Entrée	ResetCodeView -> NewPasswordView
Sorties	Succès reset -> fermeture
Retour	Géré par la navigation parente
Conditions	Succès dépend d'un reset valide

API

Domaine	Méthode	Endpoint	Description
Authentification	POST	/api/auth/reset/verify	Vérification du code de reset et récupération du token

Règles de gestion

ID	Règle
NPV01	Le champ **Mot de passe** est obligatoire.
NPV02	Le nouveau mot de passe est doit respecter les règles d'inscription.
NPV03	Le champ **Confirmation** est obligatoire.
NPV04	La confirmation doit être **identique** au mot de passe.
NPV05	En cas d'erreur, affiche le **message backend** si disponible.
NPV06	La fermeture applique un **double dismiss** pour couvrir les cas NavigationStack / Sheet.

9 – Tests XCTest

Dans le cadre du développement de l'application iOS, une structure de tests unitaires a été mise en place à l'aide du framework XCTest, intégré à Xcode.

Contrairement à la partie backend, où la logique métier, les règles d'accès et les validations sont largement couvertes par des tests automatisés détaillés dans la section 7, l'application iOS joue principalement un rôle de client consommateur de l'API REST. La majorité des contrôles fonctionnels et des validations critiques étant centralisés côté serveur, les tests Swift se concentrent donc sur la logique locale propre à l'application mobile.

Les tests unitaires implémentés portent principalement sur :

- La logique des ViewModels (architecture MVVM)
- La gestion des états d'interface (chargement, succès, erreur)
- Le traitement des réponses API (mapping JSON → modèles Swift)
- La gestion de la session utilisateur (stockage et utilisation du token JWT)

L'approche adoptée privilégie la validation des happy flows (scénarios de succès), garantissant que les interactions principales entre l'application et le backend fonctionnent correctement. Les scénarios d'erreur liés aux règles métier (droits d'accès, validations, transitions de statut) sont quant à eux déjà couverts par la suite complète de tests backend.

Cette répartition des responsabilités permet d'éviter la redondance des tests tout en assurant une couverture pertinente et cohérente avec l'architecture globale du système.

Le rapport de couverture généré par Xcode permet de mesurer la part du code Swift effectivement testée et constitue un indicateur complémentaire de qualité logicielle.

9.1 – Tests de la connexion

Fichier: LoginViewModelTests

testLogin_withValidCredentials_setsIsAuthenticatedTrue

testLogin_withEmptyEmail_setsErrorMessage

testLogin_withEmptyPassword_setsErrorMessage

9.2 - Tests des mots de passe

Fichier: NewPasswordViewModelTests

Test	Résultat attendu	Fonction
Mot de passe et confirmation identique Longueur >= 8	Passed (isFormValid == true)	isFormValid_whenPasswordsMatchAndMinLength_returnsTrue
Mot de passe et confirmation différentes	Passed (isFormValid == false)	isFormValid_whenPasswordsDifferent_returnsFalse
Mot de passe et confirmation identique Longueur < 8	Passed (isFormValid == false)	isFormValid_whenTooShort_returnsFalse
Mot de passe ou confirmation vide	Passed (isFormValid == false)	isFormValid_whenPasswordEmpty_returnsFalse

Fichier: ProjectFormViewModelTests

testIsFormValid_whenFieldsFilled_returnsTrue

testSubmit_withEmptyFields_setsErrorMessage

testEditMode_withoutChanges_setsErrorMessage

9.4 - Tests User Stories

Fichier: UserStoryFormViewModelTests

testIsFormValid_whenFieldsFilled_returnsTrue

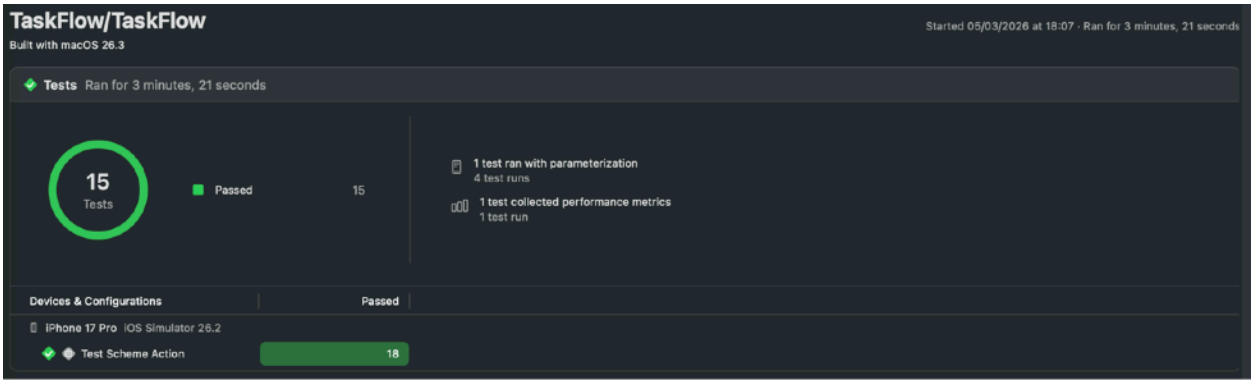
testDueAtFormatting_whenDateSet_returnsISO8601String

9.5 - Tests Session

Fichier: SessionViewModelTests

testClearSession_setsIsAuthenticatedFalse

9.6 - Résultats des tests



Name	Coverage	Executable Lines
TaskFlow.app	50,7 %	16 801
AppNavigationTitle.swift	100,0 %	75
OwnerBadgeView.swift	100,0 %	31
DraggableStory.swift	100,0 %	3
TitreView.swift	100,0 %	7
ProfileService.swift	100,0 %	13
SousTitreView.swift	100,0 %	13
RefreshService.swift	100,0 %	25
APIProject.swift	100,0 %	14
MainView.swift	100,0 %	375
ConnexionView.swift	100,0 %	199
BoutonView.swift	100,0 %	38
TagFilterChip.swift	100,0 %	53
WelcomeView.swift	100,0 %	92
Fields.swift	100,0 %	85
PasswordRulesView.swift	100,0 %	20
ProfileView.swift	99,4 %	528
ProjectsRowView.swift	98,5 %	204
TaskFlowApp.swift	98,0 %	99
ProjectCreationView.swift	97,9 %	419
UserDetailView.swift	97,7 %	699
SessionManager.swift	96,4 %	55
ProjectsView.swift	95,5 %	199
ProjectDetailView.swift	93,8 %	659
KeychainService.swift	93,7 %	95
UsersListView.swift	93,5 %	200
BoutonImageView.swift	92,8 %	111
ForgotPasswordView.swift	92,1 %	330